LINFO2364 Mining Patterns in Data

Issambre L'Hermite Dumont

This summary may not be up-to-date, the newer version is available at this address:
https://github.com/SimonDesmidt/Syntheses

Academic year 2025-2026 - Q2

UCLouvain

Contents

Introduction

In our data-driven world, the ability to extract meaningful information from vast datasets is crucial. Understanding all the aspect of this discipline is essential to derive those meaningful information, and this is the goal of this course. First, we need to define some key concepts.

1.1 Definitions

Definition 1.1. A pattern is a recurring structure in a dataset.

Patterns can be simple or complex, relevant or irrelevant. Their advantages is that they are interpretable. When found, relevant patterns can be used to make predictions, to understand the underlying structure of the data, and to make informed decisions.

Definition 1.2. Data mining is the process of discovering interesting patterns, models, and other kinds of knowledge in large data sets.

1.1.1 Type of data

We can mine data out of various types of structure of data:

- **Tabular data**: Data is organized in rows and columns. Example: spreadsheets, databases.
- **Sequences**: Data points are ordered in a sequence. Example: DNA sequences, text data.
- **Graphs, trees, networks**: Data is represented as nodes and edges. Example: social networks, web graphs.

Those structures can be discretes, continuous, enumerable data, etc. Those structures, can be combined to form more complex data types. And they can be highly structured, semi-structured, or unstructured.

Definition 1.3. Highly structured data are relational databases, with uniform record or table-like structures, with a fixed set of well-defined attributes. This is rarely the case in real-world data.

Definition 1.4. Semi-structured data are not as structured as in relational databases, but presents some structure with clearly defined semantic meaning. For example:

- Transactional dataset: structured into transactions, but each transaction is an unstructured set of values
- Sequence data set: unstructured collection of ordered sequences of values
- Graphs: set of nodes connected by a set of edges, with edges labelled given some semantic

Definition 1.5. Unstructured data have no predefined structure or organization. For example: text documents, images, audio files, videos.

Those requires advanced techniques to extract patterns, like deep learning or domain-specific methods. We can also categorize data based on the way they are generated:

- **Stored data**: Data is collected and stored in a finite set.
- **Streamed data**: Data is continuously generated and updated over time. Example: video surveillance, etc.

1.1.2 Types of data mining

Techniques and algorithms may vary depending on the data but also on way we will use mined patterns. We can categorize data mining tasks into two main types:

- **Descriptive data mining**: finds patterns that characterize the properties of the data.
- **Predictive data mining**: finds patterns that can be used to make predictions by induction.

We can identify patterns by multiples ways:

- **Frequent patterns**: patterns that identify a recurring structure in the data.
- **Associations patterns**: patterns that show rules of implications between different attributes
- **Correlations**: patterns that has positive or negative correlation between different attributes.

We can uses patterns for predictions:

- **Classification**: assign new data to classes based on similarities with historic data.
- **Regression**: predict numerical values based on the new data and the historic data.
- **Feature selection**: identify the most relevant features.

Preprocessing

To analyse data, and retrieve meaningful patterns, data often needs to be preprocessed. This step is crucial as raw data is often incomplete, inconsistent, and noisy. Preprocessing improves the quality of the data and the efficiency of the mining algorithms.

2.1 Useful statistical values

Definition 2.1. The mean (or average) of a dataset is the sum of all values divided by the number of values.

$$\bar{x} = \frac{1}{N}\sum_{i=1}^{N} x_i \tag{2.1}$$

Definition 2.2. The midrange is the average of the largest and smallest values in the set.

Definition 2.3. The variance of a dataset measures is defined like this:

$$\sigma^2 = \frac{1}{N}\sum_{i=1}^{N}(x_i - \bar{x})^2 \tag{2.2}$$

Definition 2.4. The standard deviation of a dataset is the square root of the variance:

$$\sigma = \sqrt{\sigma^2} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i - \bar{x})^2} \tag{2.3}$$

Definition 2.5. The mode for a set of data is the value that occurs most frequently. When there are multiple values with the same highest frequency, the dataset can be unimodal, bimodal, trimodal or multimodal.

Definition 2.6. Quantiles are points taken at regular intervals of a data distribution, dividing it into essentially equal-sized consecutive sets. The kth q-quantile for a given data distribution is the value x such that at most $\frac{k}{q}$ of the data values are less than x and at most $\frac{q-k}{q}$ of the data values are more than x, where k is an integer such that $0 < k < q$. There are $q - 1$ q-quantiles.

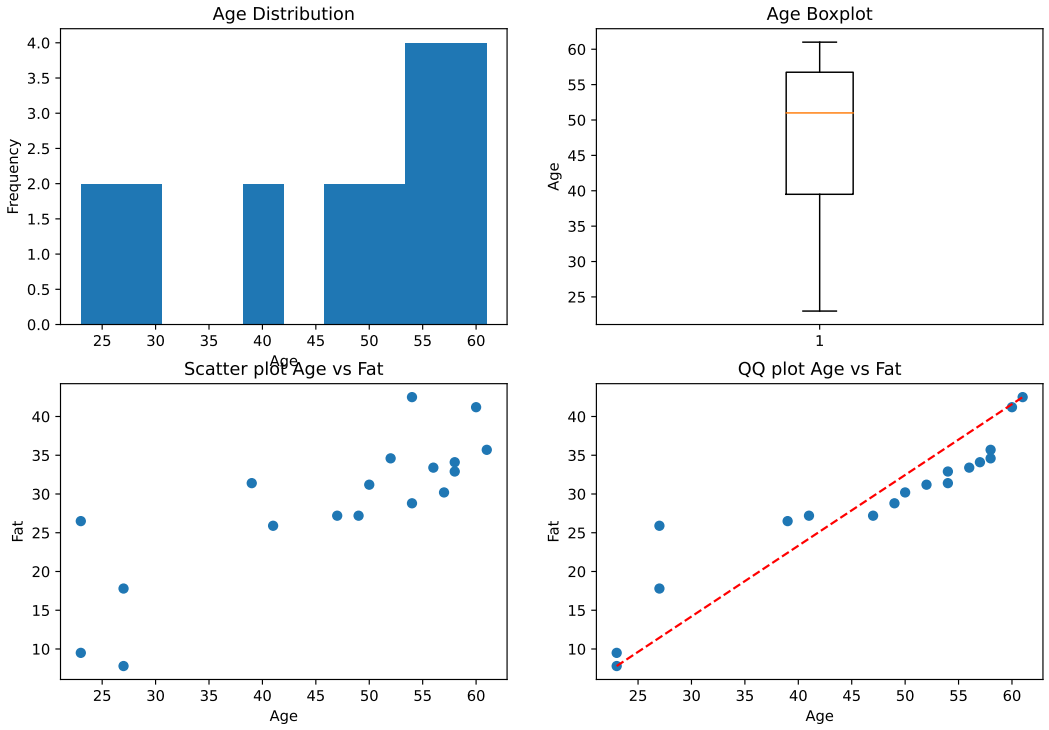
Usually we use quartiles ($q = 4$), where Q_2 is the median, or percentiles ($q = 100$).

2.2 Data plot

Plotting the data can be useful to observe some characteristics of the dataset. we can represent the data under multiples forms:

- **Histogram** is a bar chart representing the count of values present in each bucket. The buckets can be defined for each values, for ranges of values (equal-width) or for buckets containing the same number of values (equal-frequency).
- **Boxplots** are a popular way of visualizing a distribution. Typically, the ends of the box are at the quartiles so that the box length is the interquartile range. The median is marked by a line within the box. Two lines (called whiskers) outside the box extend to the smallest (minimum) and largest (maximum) observations. To highlight outliers, the whiskers are usually extended to extreme low and high observations only if these values are less than 1.5 times the interquartile range beyond the quartiles. Otherwise, the whiskers terminate at the most extreme observations occurring within 1.5 times the interquartile range.
- **Scatter plot**, each pair of values is treated as a pair of coordinates in an algebraic sense and plotted as points in the plane. It helps visually determine whether there is a relationship, pattern, or trend between the two numeric attributes.
- **Quantile-quantile plot**, or q-q plots are graphs that plot the quantiles of one univariate distribution against the corresponding quantiles of another.

Here is an example of the four plots for the same dataset:



2.3 Distances and similarities

To obtain relation between data, we can use distance and similarity measures. Distance measures the dissimilarity between two data points, while similarity measures the closeness between them. The choice of distance or similarity measure depends on the type of data and the specific problem at hand.

Definition 2.7. The norm of a vector x is:

$$||x|| = \sqrt{\sum_{i=1}^{n} x_i^2} \tag{2.4}$$

Definition 2.8. The Minkowski distance between two vectors x and y is:

$$d(x,y) = \sqrt[p]{\sum_{i=1}^{n} |x_i - y_i|^p} \tag{2.5}$$

It is a generalization of the Euclidean distance ($p = 2$) and the Manhattan distance ($p = 1$).

Definition 2.9. The supremum distance (Minkowski when $p \to \infty$) between two vectors x and y is:

$$d(x,y) = \max_i |x_i - y_i| \tag{2.6}$$

So two vectors are similar if their distances is close to 0.

Definition 2.10. The cosine similarity between two vectors x and y is:

$$s(x,y) = \frac{x \cdot y}{||x|| \cdot ||y||} = \frac{\sum_{i=1}^{n} x_i y_i}{\sqrt{\sum_{i=1}^{n} x_i^2} \cdot \sqrt{\sum_{i=1}^{n} y_i^2}} \tag{2.7}$$

Here two vectors are similar if their cosine similarity is close to 1, and dissimilar if it is close to 0.

2.4 Preprocessing techniques

First we can smooth data using binning methods. Binning methods smooth a sorted data value by consulting its neighborhood. We can smooth using bins by partitioning the sorted data into a number of equal-frequency or equal-width bins, and then replacing each data value in a bin by the mean, median, or boundary values of the bin. Another type of method to manipulate data is to use normalization. There exists multiple normalization techniques:

- **Vector normalization**: scales a vector x by it's norm to have a unit norm vector:

$$x' = \frac{x}{||x||} \tag{2.8}$$

- **Min-max normalization:** scales the vector linearly to fit in a specific range $[new\_min, new\_max]$

$$x' = \frac{(x - min(x))(new\_max - new\_min)}{max(x) - min(x)} + new\_min \tag{2.9}$$

- **Z-score normalization:** (or standardization) rescales the data to have a mean of 0 and a standard deviation of 1:

$$x' = \frac{x - \bar{x}}{\sigma} \tag{2.10}$$

- **Normalization by decimal scaling**:

$$x' = \frac{x}{10^j} \quad \text{where } j \text{ is the smallest integer such that } \max(|x'|) < 1 \tag{2.11}$$

  j can be computed as $j = \lceil \log_{10}(max(|x|)) + \epsilon \rceil$ with ϵ a small value to assure $\max(|x'|) < 1$.

For missing values, we can (last 3 methods introduce bias in the data):

- Ignore the tuple: usually done when the class label is missing or when multiple values are missing in the same tuple.
- Fill the missing value manually: time-consuming if lots of missing data
- Use a global constant (Unknown or $-\infty$): mining algorithms might mistakenly think that they form an interesting concept by having this "missing" value in common
- Use a measure of the central tendency of the attribute (e.g., mean or median)
- Use the measure of the central tendency for all samples belonging to the same class
- Use the most probable value to fill: using regression, inference-based tools, or decision tree induction.

We can also reduce the size of the data using sampling:

- **SRSWOR** (Simple random sample without replacement of size s): this sampling is created by drawing samples from D, and every time a sample is drawn, it is not to be placed back into the data set D.
- **SRSWR** (Simple random sample with replacement of size s): this sampling is similar to SRSWOR, except that when a sample is drawn, it can be redrawn again, potentially.
- **Stratified sampling**: If D is divided into mutually disjoint parts called strata, a stratified sample of D is generated by obtaining a sample at each stratum. This helps ensure a representative sample, each stratum need to be defined manually.

Itemset mining

3.1 Definitions

Definition 3.1. An itemset is a set of items and an itemset containing k items is called a k-itemset.

Frequent itemset mining finds associations and correlations between items in large transactional or relational datasets. It is widely used in market basket analysis for example. It is the analysis of the buying habits of customers by finding associations between the different items that customers place in their shopping basket.

Definition 3.2. Itemset Y is called a **superitemset** of X if X is a **subitemset** of Y, i.e. if $X \subset Y$. Every items of X is in Y but at least one item in Y does not appear in X.

Definition 3.3. An itemset X is **closed** in $\mathcal{D}$ if there exists no proper superitemset Y such that Y has the same support count as X in $\mathcal{D}$.

Definition 3.4. A transactional dataset is a collection of transactions, where each transaction is a set of items.

With $\mathcal{L}$ the set of possible items, $\mathcal{T}$ the set of possible transactions, a transactional dataset $\mathcal{D}$ can be seen as the function $\mathcal{D} : \mathcal{T} \to 2^{\mathcal{L}}$. It can also be represented as a binary matrix, where rows represent transactions and columns represent items, mathematically it is like the function $\mathcal{D} : \mathcal{T} \times \mathcal{L} \to \{0, 1\}$.

Definition 3.5. An itemset $l \subset \mathcal{L}$ covers (or matches) a transaction $t \subset \mathcal{T}$ if every item from l is in t. Consider the match function, with $D(t, l)$ the function representing the transactional dataset:

$$\texttt{match}(l, t) = \begin{cases} 1 & \text{if } \forall i \in l, D(t, i) = 1 \\ 0 & \text{otherwise} \end{cases} \tag{3.1}$$

Definition 3.6. The occurrence frequency of an itemset (also called frequency, support count, count or absolute support) is the number of transactions that contain the itemset (i.e., the size of the cover)

$$\texttt{support}_{\texttt{count}}(l) = \sum_{t \in \mathcal{T}} \texttt{match}(l, t) \tag{3.2}$$

An itemset is called frequent if its support count is greater than a minimum support count threshold θ. If an itemset is frequent then all of its subsets are also frequent.

Definition 3.7. The relative support of an itemset l in a database $\mathcal{D}$ is the percentage of transactions containing the itemset:

$$\texttt{support}_{\mathcal{D}}(l) = \frac{\texttt{support}_{\texttt{count}}(l)}{|\mathcal{T}|} \tag{3.3}$$

Definition 3.8. An itemset X is a **maximal frequent itemset** in $\mathcal{D}$ if X is frequent, and there exists no superitemset Y such that Y is frequent in $\mathcal{D}$.

Definition 3.9. An association rule represents an implication of the form $X \Rightarrow Y$, where X and Y are itemsets.

Definition 3.10. The rule support is the support of the itemset containing all the elements of the rule $\texttt{support}_{\texttt{count}}(X \Rightarrow Y) = \texttt{support}_{\texttt{count}}(X \cup Y)$.

We consider an association rule or an itemset as frequent if its support is greater than a user-defined minimum support threshold θ.

Definition 3.11. The rule confidence is the percentage of transactions containing X that contains Y too

$$\texttt{confidence}(X \Rightarrow Y) = P(Y|X) = \frac{\texttt{support}_{\texttt{count}}(X \cup Y)}{\texttt{support}_{\texttt{count}}(X)} \tag{3.4}$$

3.2 Apriori algorithm

The key idea behind the Apriori algorithm is that all nonempty subsets of a frequent itemset must also be frequent.

Algorithm 1 Apriori algorithm

Input: Transactional dataset $\mathcal{D}$, minimum support threshold θ
Output: Set F of frequent itemsets
$F = \emptyset$
$C_1 = \{\{i\} : i \in \mathcal{L}\}$
$L_1 = \{c \in C_1 : \texttt{support}_{\texttt{count}}(c) \geq \theta\}$
$F = F \cup L_1$
$k = 2$
while L_k is not empty **do**
    $C_k = \texttt{apriori\_gen}(L_{k-1})$
    $L_k = \{c \in C_k : \texttt{support}_{\texttt{count}}(c) \geq \theta\}$
    $F = F \cup L_k$
    $k = k + 1$
end while
return F

The `apriori_gen`(L_{k-1}) function generates candidate k-itemsets from the frequent (k-1)-itemsets L_{k-1} by joining them and pruning those that have infrequent subsets.

Algorithm 2 apriori_gen(L_{k-1})

Input: Set L_{k-1} of frequent (k-1)-itemsets
Output: Set C_k of candidate k-itemsets
$C_k = \emptyset$
for each $l_1 \in L_{k-1}$ **do**
    for each $l_2 \in L_{k-1}$ **do**
        if $l_1[:k-2] = l_2[:k-2]$ and $l_1[k-1] < l_2[k-1]$ **then**
            $c = l_1 \cup l_2$
            if all (k-1)-subsets of c are in L_{k-1} **then**
                $C_k = C_k \cup \{c\}$
            end if
        end if
    end for
end for
return C_k

The number of candidate itemsets is bounded by $\mathcal{O}\left(\binom{|\mathcal{L}|}{k}\right)$.
The two mains weaknesses of the Apriori algorithm are:

- Potential generation of huge candidate sets
- Repeated scan of the database and checks for pattern matching to the transactions

The Apriori algorithm can be improved using:

- **Hash-based techniques**: Instead of building L_2 from the join operation, build it similarly to L_1. When scanning the transactions, generate the 2-itemset subset of the transaction, hash them and map them to corresponding buckets containings counters to be increased. L_2 is the set of the 2-itemsets associated to buckets of a count higher than θ
- **Transaction reduction**: A transaction that does not contains any frequent k-itemsets cannot contain any frequent $(k+1)$-itemsets. They can be flagged and avoided at next steps.
- **Partitioning**: The dataset can be partitioned in N non-overlapping sub-database. Given θ, the minimum support count threshold for the whole database, $\theta' = \lfloor \frac{\theta}{N} \rfloor$ is the threshold for each partition. The candidate set C_k is the union of all local frequent k-itemset of each partition as any itemset that is potentially frequent with respect to D must occur as a frequent itemset in at least one of the partitions.
- **Sampling**: The frequent itemsets of a sample of the database are computed to serve as C_k. Some degree of accuracy is traded for eciency, thus a lower value than θ' is used to counterbalanced the effect.

3.3 FP-growth algorithm

For the FP-growth algorithm, we first need to create the FP-tree, which is a tree structure that represents all of the transactions in the database.

Algorithm 3 `fp_growth`($\mathcal{D}$, θ)

Input: Transactional dataset $\mathcal{D}$, minimum support threshold θ
Output: Set F of frequent itemsets
tree = `fp_tree`($\mathcal{D}$)
F = `fp_growth`(tree, θ)
return F

First we need to build the FP-tree, which is done like this:

Algorithm 4 `fp_tree`($\mathcal{D}$, θ)

Input: Transactional dataset $\mathcal{D}$, minimum support threshold θ
Output: A FP-tree
Scan D and compute the support count of each item, and keep only those with support count $\geq \theta$ in F
Sort F in descending order of support count, creating L the list of frequent items
Create the root of the tree
for each transaction $\tau \in \mathcal{D}$ **do**
    Sort the items of τ following L
    Add the transaction τ to the tree, by incrementing counters along the way and creating new nodes is the path is not present
end for
return Tree

Then we can mine the FP-tree to find all the frequent itemsets. The mining algorithm, works like this:

Algorithm 5 `fp_growth_mining`(Tree, α)

Input: The FP-tree and a suffix α (initially empty)
Output: Set F of frequent itemsets
if Tree contains a single path P **then**
    for each combination (denoted as β) of the nodes in the path P **do**
        generate pattern $\beta \cup \alpha$ with $\texttt{support}_{\texttt{count}}$ = min($\texttt{support}_{\texttt{count}}$ of nodes $\in \beta$)
    end for
else
    for each a_i in the header of Tree **do**
        generate pattern $\beta = a_i \cup \alpha$ with $\texttt{support}_{\texttt{count}} = \texttt{support}_{\texttt{count}}(a_i)$
        construct β's conditional pattern base and then β's conditional FP-tree $Tree_\beta$
        if $Tree_\beta \neq \emptyset$ **then**
            `fp_growth_mining`($Tree_\beta$, β)
        end if
    end for
end if

And after mining the FP-tree, we will get all the frequent itemsets.

3.4 Eclat algorithm

This algorithm works with a vertical data format, it means that we works with sets of transactions for each item, instead of sets of items for each transaction. The key idea is to use the intersection of transaction sets to compute the support count of itemsets. We can write the algorithm like this:

Algorithm 6 `eclat_algo`($\mathcal{D}, \theta$)

Input: Transactional dataset $\mathcal{D}$, minimum support threshold θ
Output: Set F of frequent itemsets
Transform $\mathcal{D}$ into a vertical data format
$F = \emptyset$
$F \cup \{i : \text{if size}(t(i)) \geq \theta\}$ for all item transaction sets $t(i)$ of item i
$C = F$
while C is not empty **do**
    $new\_C = \emptyset$
    for each pair of transaction sets A and B in C **do**
        c = $A \cap B$
        if size$(c) \geq \theta$ **then**
            $new\_C \cup c$
        end if
    end for
    $F = F \cup new\_C$
    $C = new\_C$
end while
return F

3.5 Association rule mining

Algorithm 7 `Association_rule_mining_algo`$(\mathcal{D}, \theta, \epsilon)$

Input: Transactional dataset $\mathcal{D}$, minimum support threshold θ, minimum confidence threshold ϵ
Output: Set FC of frequent and confident itemsets
$F =$ `apriori_algo`$(\mathcal{D}, \theta)$ (OR other frequent itemset mining algorithm)
$FC = \emptyset$
Generate set S the powerset ($\backslash\{\emptyset\}$) of F (i.e., all non empty subsets of F)
for each itemset $s \in S$ **do**
    Compute confidence of the rule $s \rightarrow (F \backslash s)$ using the support counts of s and F:

$$\texttt{confidence}(s \rightarrow (F \backslash s)) = \frac{\texttt{support}_{\texttt{count}}(F)}{\texttt{support}_{\texttt{count}}(s)} \tag{3.5}$$

    if confidence $\geq \epsilon$ **then**
        $FC = FC \cup \{s\}$
    end if
end for
return FC

3.6 Correlation measures

Even when we observe strong associations rules between items, they might not be interesting. To decide that, we can either manually chose if the pattern is interesting or not, it is called **Subjective assessement**. Or we can use **Objective assessment** which is based on statistics. For that, we can introduces the following measures:

Definition 3.12. The lift of a pair of item analyses the dependence and the correlation between them. It is defined as:

$$\texttt{lift}(A, B) = \frac{P(A \cup B)}{P(A)P(B)} = \frac{P(B|A)}{P(B)} = \frac{conf(A \Rightarrow B)}{P(B)} \tag{3.6}$$

- If $\texttt{lift}(A, B) < 1$, A and B are negatively correlated, i.e., the occurence of one likely leads to the absence of the other
- If $\texttt{lift}(A, B) = 1$, A and B are independant
- If $\texttt{lift}(A, B) > 1$, A and B are positively correlated, i.e., the occurence of one implies the occurence of the other

Definition 3.13. Based on the contingency table of the two items, we can compute the χ^2 **(chi-square)** statistics that tests the independence of the two items. The test is based on statistics table, and degrees of freedom. The degrees of freedom can be computed with:

$$DF = (r - 1)(c - 1) \tag{3.7}$$

And the χ^2 metrics is computed with:

$$\chi^2 = \sum_{i=1}^{r} \sum_{j=1}^{c} \frac{(o_{ij} - e_{ij})^2}{e_{ij}} \tag{3.8}$$

with o_{ij} the observed frequency of the event (A_i, B_j) and e_{ij} the expected frequency of the event (A_i, B_j). e_{ij} is computed as:

$$e_{i,j} = \frac{\texttt{count}(A = a_i) \times \texttt{count}(B = b_j)}{Total} \tag{3.9}$$

with n the number of data tuples.

Example of a contingency table for two items A and B:

$o_{i,j}$	A	$\neg A$	Total
B	4000	3500	7500
$\neg B$	2000	500	2500
Total	6000	4000	10000

And the expected table is:

$e_{i,j}$	A	$\neg A$
B	4500	3000
$\neg B$	1500	1000

And here is an example of the χ^2, for different confidence values of the pair (A, B):

	P										
DF	**0.995**	**0.975**	**0.20**	**0.10**	**0.05**	**0.025**	**0.02**	**0.01**	**0.005**	**0.002**	**0.001**
1	0.0000393	0.000982	1.642	2.706	3.841	5.024	5.412	6.635	7.879	9.550	10.828
2	0.0100	0.0506	3.219	4.605	5.991	7.378	7.824	9.210	10.597	12.429	13.816
3	0.0717	0.216	4.642	6.251	7.815	9.348	9.837	11.345	12.838	14.796	16.266
4	0.207	0.484	5.989	7.779	9.488	11.143	11.668	13.277	14.860	16.924	18.467

Figure 3.1: Example of χ^2 table

Definition 3.14. All confidence measure is the minimum confidence of the two association rules related to A and B ($A \Rightarrow B$ and $B \Rightarrow A$). And it is defined as:

$$\begin{aligned} \texttt{all\_conf}(A, B) &= \frac{\texttt{support}_{\texttt{count}}(A \cup B)}{\max\{\texttt{support}_{\texttt{count}}(A), \texttt{support}_{\texttt{count}}(B)\}} \\ &= \min\{P(B|A), P(A|B)\} \end{aligned} \tag{3.10}$$

Definition 3.15. Max confidence measure is also the maximum confidence of the two association rules related to A and B ($A \Rightarrow B$ and $B \Rightarrow A$). And it is defined as:

$$\texttt{max\_conf}(A, B) = \max\{P(B|A), P(A|B)\} \tag{3.11}$$

Definition 3.16. Kulczynski measure can be view as an average confidence of the two association rules related to A and B ($A \Rightarrow B$ and $B \Rightarrow A$). And it is defined as:

$$\texttt{kulc}(A, B) = \frac{1}{2}(P(A|B) + P(B|A)) \tag{3.12}$$

Definition 3.17. Cosine measure can be view as a harmonized lift measure and is defined as:

$$\begin{aligned} \texttt{cosine}(A,B) &= \frac{\texttt{support}_{\text{count}}(A \cup B)}{\sqrt{\texttt{support}_{\text{count}}(A)\texttt{support}_{\text{count}}(B)}} \\ &= \frac{P(A \cup B)}{\sqrt{P(A)P(B)}} = \sqrt{P(A|B)P(B|A)} \end{aligned} \tag{3.13}$$

For all thoses measures, from the all confidence to the cosine measure, the measures are in the range $[0,1]$ and they can be interpreted as:

- If $\texttt{cosine}(A,B) < 0.5$, A and B are negatively correlated
- If $\texttt{cosine}(A,B) = 0.5$, A and B are independant
- If $\texttt{cosine}(A,B) > 0.5$, A and B are positively correlated

To decide whether two items are correlated or not, we can use all the previous mesures. We can also check the relation between two items by testing their conditional probabilities (i.e. $P(A|B)$ and $P(B|A)$). And with that, we can introduce the imbalance ratio.

Definition 3.18. The imbalance ratio measures the difference between the two conditional probabilities and return a value near 0 if the two probabilities are close and otherwise the closer it is to 1, the more imbalanced the two probabilities are. It is defined as:

$$\texttt{IR(A,B)} = \frac{|\texttt{support}_{\text{count}}(A) - \texttt{support}_{\text{count}}(B)|}{\texttt{support}_{\text{count}}(A) + \texttt{support}_{\text{count}}(B) - \texttt{support}_{\text{count}}(A \cup B)} \tag{3.14}$$

Usually it is advised to use kulc and the imbalance ratio together to decide.

3.7 Multilevel associations and mining rare patterns

3.7.1 Multilevel associations mining

Items can have multiple level of abstraction. For example, a store can sell different type of goods (fruits, vegetables, drinks, etc) and in those categories, there can be different subcategories (for drink: alcool, softs, milks, etc) and in those subcategories, we can have different items. Patterns can be mined at each level of abstraction, and usually patterns at more detailled levels are the most interesting. From the levels of abstraction, we can draw a tree (which is not unique), to show a way of categorizing the items. For example, we can have this concept hierarchy:

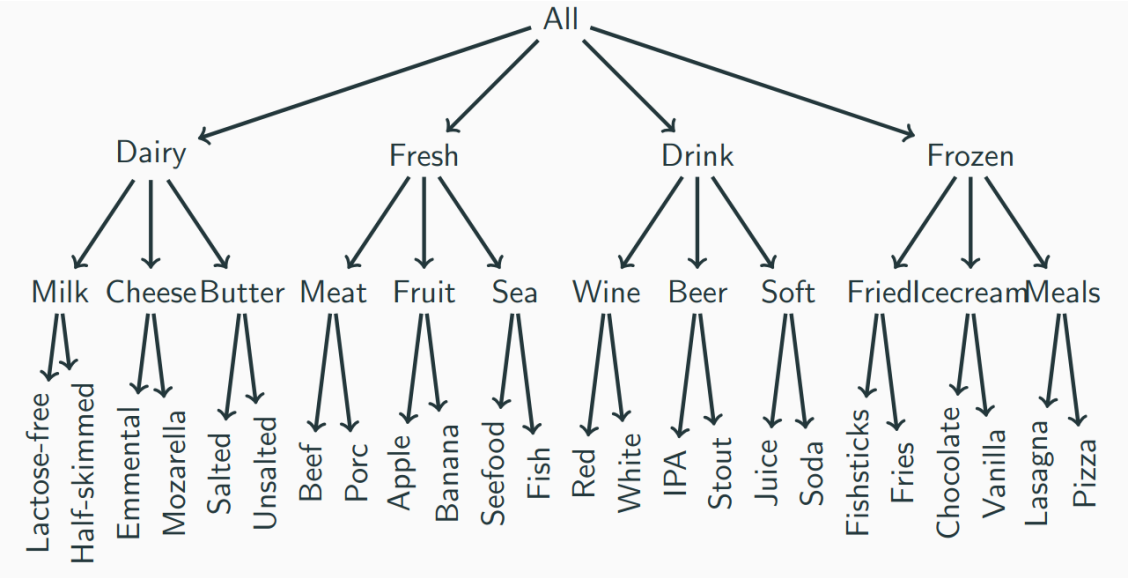


Figure 3.2: Example of a concept hierarchy

We can mine multilevel association rules by using 3 differents types of support thresholds:

Definition 3.19. Uniform support mining uses the same minimum support threshold for all the levels of the hierarchy. With this type of support, we do not consider an itemset without a frequent ancestor.

Definition 3.20. Reduced support mining uses a lower minimum support threshold for the lower levels of the hierarchy. And so with this, each level as it's own minimum support threshold, and an item might be frequent even though its ancestor is not frequent.

Definition 3.21. Group-based support mining uses specialized threshold for each group of items. For example, low price value items have a higher threshold, high price value items have a lower threshold. Usually, the mining is done with the lower support, followed by post-processing step to remove itemset that do not meet their own categories threshold

3.7.2 Mining rare patterns

Rare patterns or patterns that reflect a negative correlation between items are sometimes interesting to find.

Definition 3.22. A **negatively correlated pattern** is a pattern $A \cup B$, such that A and B are frequent but they rarely occur together, which means that:

$$\texttt{support}_{\texttt{count}}(A \cup B) < \texttt{support}_{\texttt{count}}(A)\texttt{support}_{\texttt{count}}(B) \tag{3.15}$$

And thus, A and B are negatively correlated.

Definition 3.23. A **strongly negatively correlated pattern** is a pattern $A \cup B$, such that A and B are frequent but they rarely occur together, which means that:

$$\texttt{support}_{\texttt{count}}(A \cup B) \ll \texttt{support}_{\texttt{count}}(A)\texttt{support}_{\texttt{count}}(B) \tag{3.16}$$

And thus, A and B are strongly negatively correlated.

But the formula is not null invariant, another way to check if two items are strongly negatively correlated is:

$$\begin{aligned}\texttt{support}_{\text{count}}(A \cup \neg B)\texttt{support}_{\text{count}}(\neg A \cup B) \gg \\ \texttt{support}_{\text{count}}(A \cup B)\texttt{support}_{\text{count}}(\neg A \cup \neg B)\end{aligned} \tag{3.17}$$

But is also not null invariant. To really have a null invariant measure, we can use the Kulczynski measure:

$$\frac{1}{2}(P(A|B) + P(B|A)) < \delta \tag{3.18}$$

with δ a user-defined negative pattern threshold.

3.8 Constraint-based pattern mining

In addition to the suppport, it can be interesting to find patterns that satisfies some constraints. For example, we can be interested in finding patterns that contains a specific item, or patterns with a total value under a specific budget (for a supermarket for example). There exists different types of constraints:

- **Knowledge** type constraints: specify the type of knowledge to be mined (e.g., association, correlation, classification, clustering,...)
- **Data** constraints: set of task-relevant data (e.g., sequence mining, graph mining,...)
- **Dimension/level** constraints: specify the desired dimensions of the data, the abstraction levels, the level of the concept hierarchies
- **Interestingness** constraints: specify the thresholds on statistical measures (e.g., support, confidence, correlation)
- **Rule/pattern** constraints: specify the form of, or conditions on, the rules/patterns to be mined (e.g., max/min number of items, templates, relations between attributes)

Thoses different types of constraints can be categorized in 3 different categories with different properties.

Definition 3.24. A constraint C is **pattern antimonotonic**, if an itemset does not satisfy constraint C, none of its supersets will satisfy C (e.g., the support constraint)

When a constraint is pattern antimonotonic, we can prune the search space by not considering the supersets of an itemset that does not satisfy the constraint.

Definition 3.25. A constraint C is **pattern monotonic** if an itemset satisfy constraint C, all of its supersets will satisfy C.

In opposition to pattern antimonotonic constraints, when a constraint is pattern monotonic, we cannot prune the search space, it is only a constraint that is useful to filter the results at the end of the mining process. It can also help to reduces the number of querries done to check if a superset satisfies the constraint because we only need to check if the superset contains an accepted set.

Definition 3.26. Convertible constraint is a constraint that can be transformed into a pattern monotonic or pattern antimonotonic constraint by reordering the items in the itemsets.

Definition 3.27. A constraint c is **succinct** if it can be enforced by directly pruning some data objects from the database (**data succinct**) or by directly enumerating all and only those sets that are guaranteed to satisfy the constraint (**pattern succinct**).

3.8.1 Common constraints and their categories

Constraints	Antimonotonic	Monotonic	Succinct
$v \in S$		$\surd$	$\surd$
$S \supseteq V$		$\surd$	$\surd$
$S \subseteq V$	$\surd$		$\surd$
$\min(S) \leq v$		$\surd$	$\surd$
$\min(S) \geq v$	$\surd$		$\surd$
$\max(S) \leq v$	$\surd$		$\surd$
$\max(S) \geq v$		$\surd$	$\surd$
$\texttt{count}(S) \leq v$	$\surd$		
$\texttt{count}(S) \geq v$		$\surd$	
$\texttt{sum}(S) \leq v (\forall a \in S, a \geq 0)$	$\surd$		
$\texttt{sum}(S) \geq v (\forall a \in S, a \geq 0)$		$\surd$	
$\texttt{range}(S) \leq v$	$\surd$		
$\texttt{range}(S) \geq v$		$\surd$	
$\texttt{avg}(S) \leq v$	convertible	convertible	
$\texttt{avg}(S) \geq v$	convertible	convertible	
$\texttt{support}(S) \leq \theta$	$\surd$		
$\texttt{support}(S) \geq \theta$		$\surd$	
$\texttt{range}(S) \leq \epsilon$	$\surd$		
$\texttt{range}(S) \geq \epsilon$		$\surd$	

3.8.2 Constraint programming approach

Constraint programming is a powerful paradigm for solving combinatorial problems, including constraint-based pattern mining. It allows us to express complex constraints and search for solutions that satisfy those constraints efficiently. In the context of constraint-based pattern mining, we are going to use 1 boolean variables for each item and transaction. And then we are going to link those variables with constraints. And we can find all the patterns that satisfy the constraints by exploring the search space of the variables and checking the constraints.
So we are going to define the variables for the items as X_i and for the transactions as Y_i. Let's define the basic constraints:

- **Cover constraint**: if we select a transaction then we need at least to select all items in that transaction.

$$Y_m = 1 \Leftrightarrow \sum_k X_k \times \mathcal{D}_{\tau_m, i_k} = \sum_k X_k \tag{3.19}$$

But we can also write it as:

$$Y_m = 1 \Leftrightarrow \sum_k X_k \times (1 - \mathcal{D}_{\tau_m, i_k}) = 0 \tag{3.20}$$

- **Minimum support constraint**: we need to select at least θ transactions.

$$\sum_m Y_m \geq \theta \tag{3.21}$$

With those, we can find all the frequent itemsets. But we can also add other constraints to find more specific patterns:

- **Sum of values constraint**: we want that the sum of the values of the items selected respects a budget. (works also with minimum value of the sum)

$$\sum_k X_k \times v_k \leq P \tag{3.22}$$

- **Maximum values constraint**: we want that the maximum value of the items selected is under a specific threshold.

$$\max_k X_k \times v_k \leq P \tag{3.23}$$

And finally, if we want to find patterns that are not only frequent but also maximal or closed, we can use the following constraints:

- **Maximal patterns constraint**:

$$X_k = 1 \Leftrightarrow \sum_m Y_m \times \mathcal{D}_{\tau_m, i_k} \geq \theta \tag{3.24}$$

- **Closed patterns constraint**:

$$X_k = 1 \Leftrightarrow \sum_m Y_m \times \mathcal{D}_{\tau_m, i_k} = \sum_m Y_m \tag{3.25}$$

But it can also be written as:

$$X_k = 1 \Leftrightarrow \sum_m Y_m \times (1 - \mathcal{D}_{\tau_m, i_k}) = 0 \tag{3.26}$$

After defining the variables and the constraints, we can use a constraint solver to find all the patterns that satisfy the constraints. The solver will explore the search space of the variables and check the constraints to find all the solutions that satisfy them. We can do it by hand by exploring the tree of possible assignments values of the variables and checking the constraints at each node of the tree. Here is an example of a search tree for a simple dataset with 4 items.

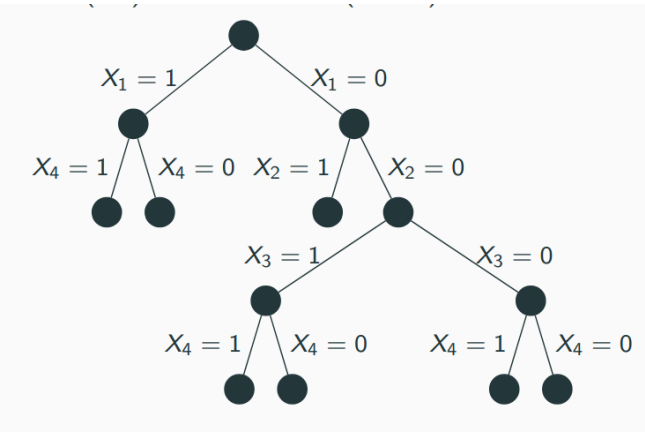


Figure 3.3: Example of a search tree

Definition 3.28. The CoverSize propagator is a constraint that enforces consistency between a set of item and a cardinality constraint (e.g., minimum support). It is defined as $\texttt{CoverSize}(X, N, D)$ where X is an array of boolean ($D(X_k) = \{0, 1\}$) for each item, N is an integer variable that represents the number of transactions covered by the selected items ($D(N) = \{0, \ldots, T\}$), and D is the dataset. With that the minimum support constraint becomes $N \geq \theta$

3.9 Sequence mining

Definition 3.29. A **sequence** is an ordered (w.r.t time) list of events and is noted $\langle e_1 e_2 \ldots e_n \rangle$, where e_i is an event (called an **element** of the sequence). An event can be a single item or a set of items that occur together at a specific time.

We consider that an item can occur at most once in an event of a sequence, but can occur multiple times in different events of a sequence.

Definition 3.30. The **length** of a sequence is the number of items in the sequence.

For example, the length of the sequence $\langle a(bc)c \rangle$ is 4, because it contains 4 items: a, b, c and d. Even though, there is 3 elements. A sequence with l elements is called a **l-sequence**.

Definition 3.31. A sequence $\alpha = \langle a_1 a_2 \ldots a_n \rangle$ is called a **subsequence** of another sequence $\beta = \langle b_1 b_2 \ldots b_m \rangle$, and β is a **supersequence** of α, denoted as $\alpha \sqsubseteq \beta$, if there exists integers $1 \leq j_1 < j_2 < \cdots < j_n \leq m$ such that $a_1 \subseteq b_{j_1}, a_2 \subseteq b_{j_2}, \ldots, a_n \subseteq b_{j_n}$.

Definition 3.32. A **sequence database** S is a set of tuples $\langle SID, s \rangle$, where SID is a sequence ID and s is a sequence.

A tuple $\langle SID, s \rangle$ is said to **contain** a sequence α if $\alpha \sqsubseteq s$.

Definition 3.33. The **support** of a sequence α in a sequence database S is the number of tuples in S that contain α.

$$\texttt{support}_S(\alpha) = |\{\langle SID, s \rangle | (\langle SID, s \rangle \in S) \wedge (\alpha \sqsubseteq s)\}| \tag{3.27}$$

As usual a sequence is **frequent** in sequence database S if its support is no less than a minimum support threshold θ ($\texttt{support}_S(\alpha) \geq \theta$). A frequent sequence is called a **sequential pattern** and an **l-pattern** is a sequential pattern of length l.

3.9.1 Generalize Sequential Patterns (GSP)

Based on the Apriori algorithm, the GSP algorithm generates candidate sequences by joining two frequent sequences of size k to generate a sequence of size $k+1$, then it prunes the candidates based on the minimum support threshold and repeat.

3.9.2 Sequential Pattern Discovery using Equivalent classes (SPADE)

In the same idea of the Eclat algorithm, the SPADE algorithm uses a vertical data format to mine sequential patterns. The idea is to represent the sequence database in a vertical format, where we store $(SID, position)$, then we can compute the support of a sequence by intersecting the sets of $(SID, position)$ of its subsequences. And we explore the search space of the sequences by using a depth-first search strategy, which allows us to find all the frequent sequences in the database based on the apriori principle.

3.9.3 PrefixSpan

Definition 3.34. The **prefix** of a sequence $\alpha = \langle a_1 a_2 \dots a_n \rangle$ is defined as the sequence $\beta = \langle a_1 a_2 \dots a_k \rangle$ with $k < n$. And the **suffix** of α is defined as the sequence $\gamma = \langle a_{k+1} a_{k+2} \dots a_n \rangle$.

The mining process of sequential patterns can be decomposed into a set of subproblems:

- Given the length-1 sequential patterns $\{\langle x_1 \rangle, \langle x_2 \rangle, \dots, \langle x_n \rangle\}$, the complete set of sequential patterns in S can be partitioned into n disjoint subsets. i^{th} subset is the subset of sequences from S with $\langle x_i \rangle$ as prefix
- Given the length-k sequential pattern α and length-(k+1) sequences $\{\beta_1, \beta_2, \dots, \beta_n\}$ with prefix α, the complete set of sequential patterns with pattern α can be partitioned into n disjoint subsets. j^{th} subset is the subset of sequences with β_j as prefix

The PrefixSpan algorithm is based on the pattern growth approach, and use the idea of prefix and suffix to mine sequential patterns. The algorithm can be thus described as follows:

1. Find length-1 sequential patterns and count their support
2. Partition of the search space: patterns with prefix $\langle a \rangle$, patterns with prefix $\langle b \rangle$, ... For each partition, mine the patterns recursively.
3. For the given pattern, construct the projected database. To construct the projected database, we match the current pattern with each sequence. If the sequence match, the part matching the pattern is removed, and the remaining is kept into the projected database.

4. We mine recursively the projected database by doing:
   (a) Scan the $\langle a \rangle$-projected database for locally frequent items
   (b) Partition of the search space: patterns with prefix $\langle aa \rangle$, patterns with prefix $\langle ab \rangle$, ... For each partition, mine the patterns recursively.
   (c) Construction of the projected database
   (d) Mine recursively the projected database

As we can see, the algorithm is based on a depth-first search strategy by recursion.

3.9.4 Constraint sequential patterns

Like in the previouses sections, we can also add constraints to the sequential pattern mining process. For example, we can add a constraint on the closeness of the patterns. But there exists more specific constraints for sequential patterns, such as:

- Constraints on the duration (we want information over a maximum span of x month)
- Constraints on the length (minimum or maximum length for the sequence)
- Constraints on the event folding window w : If event occurs within the same window, they are considered occuring together
- Constraints on the time gap between events (We could be interested in sequences of close event (maximal gap) or spaced events (minimal gap))
- Constraints on the type of sequence (One could wish for some pattern matching a given regular expression)

Graph mining

4.1 Definition

Definition 4.1. A **graph** g is defined by:

- a **vertex set** $V(g)$
- an **edge set** $E(g)$ of directed or undirected edges
- a **label function** λ_v mapping each vertex to a label
- a **label function** λ_e mapping each edge to a label

Definition 4.2. A graph g is a **subgraph** of another graph g' if there exists a **subgraph isomorphism** from g to g'.

Definition 4.3. A **subgraph isomorphism** exists if and only if there exist a mapping M from g to g', such that for each vertex $v \in V(g)$ and each edge $(u, v) \in E(g)$, we have:

- $M(v) \in V(g')$
- $(M(u), M(v)) \in E(g')$
- $\lambda_V(v) = \lambda_{V'}(M(v))$
- $\lambda_E((u, v)) = \lambda_{E'}((M(u), M(v)))$

Finding subgraphs is a computationally hard problem, it is NP-complete. A graph g matches or covers a graph g' from a graph database if g is a subgraph of g'.

Definition 4.4. Given a labeled graph data set, $D = \{G_1, G_2, \ldots G_n\}$, we define $\texttt{support}(g)$ (or $\texttt{frequency}(g)$) as the percentage (or number) of graphs in a graph database D where g is a subgraph.

A frequent graph is a graph whose support is greater or equal to a minimum support threshold θ.
To find frequent subgraphs, we have two steps:

- Generate frequent substructure candidates
- Check the frequency (involves subgraph isomorphism test)

Usually we used Apriori or pattern growth based method, and we optimize the first step to avoid a maximum the subgraph isomorphism test, which is the most expensive part of the algorithm.

4.2 AprioriGraph

Based on the Apriori algorithm, here the generation of the candidate is done by adding either a vertex (with an edge to be connected to the existing graph) or an edge between two existing vertices.

Algorithm 8 AprioriGraph algorithm

Input: Transactional dataset $\mathcal{D}$, minimum support threshold θ, set of frequent subgraphs of size k (S_k)
Output: Set S_{k+1} of frequent itemsets
$S_{k+1} = \emptyset$
for each frequent $g_i \in S_k$ **do**
    for each frequent $g_j \in S_k$ **do**
        for each size $(k+1)$ graph g formed by the merge of g_i and g_j **do**
            if g is frequent in $\mathcal{D}$ and $g \notin S_{k+1}$ **then**
                $S_{k+1} = S_{k+1} \cup \{g\}$
            end if
        end for
    end for
end for
if $S_{k+1} \neq \emptyset$ **then**
    S_{k+2} = AprioriGraph($\mathcal{D}$, θ, S_{k+1})
    Return $S_{k+2} \cup S_{k+1}$
end if

4.3 PatternGrowthGraph

First let us define the notations for adding an extension (new edge e) of a graph g:

- adding a vertex: called **forward extension** $g \diamond_{xf} e$
- adding an edge between two existing vertices: called **backward extension** $g \diamond_{xb} e$

Algorithm 9 PatternGrowthGraph algorithm

Input: Seed of the graph to grow g, dataset of graphs $\mathcal{D}$, minimum support threshold θ, current set of frequent graphs S
Output: Set S of frequent itemsets
if $g \in S$ **then**
    return
else
    $S = S \cup \{g\}$
end if
Scan $\mathcal{D}$ once, find all the edges e such that g can be extended to $g \diamond_x e$
for each frequent $g \diamond_x e$ **do**
    PatternGrowthGraph($g \diamond_x e$, $\mathcal{D}$, θ, S)
end for
return S

4.4 Graph-based Substructure Pattern Mining (gSpan)

This algorithm is based on the pattern growth method, but it is designed to reduce the generation of duplicates. To do that, it uses a canonical form of the graph called **DFS code** to represent the graph. The DFS code is a sequence of tuples that represent the edges of the graph in a depth-first search order. This helps to deal with the fact that graphs does not have natural ordering and many isomorphisms.

4.4.1 DFS codes

Definition 4.5. A graph G subscripted with a DFS tree T is written G_T. T is called a **DFS subscription** of G.

Definition 4.6. Subscripted graph are then transformed into a **DFS code**. DFS code are a sequence of tuples $\langle v_i, v_j, \lambda_V(v_i), \lambda_V(v_j), \lambda_E((v_i, v_j))\rangle$ representing edges.

The construction of the DFS code follows some rules:

1. Build the DFS tree of the graph by doing a DFS and assigning a number to each vertex in the order they are visited.
2. Add edges in order of the DFS tree. There is two types of edges:
    - an edge that connects a vertex to a new vertex (not visited yet). It means they are discovered during the build of the DFS tree. They are added in the order they are discovered during the DFS.
    - an edge that connects a vertex to an already visited vertex. It means they are not discovered during the build of the DFS tree. They are added after all the forward edges.

The construction of the DFS code is deterministic, it follows some rules to ensure that the same graph will always have the same DFS code. The rules are as follows:

- the edges of the DFS tree (forward edges) are ordered in the same order as the discovery of the tree
- the backward edges are ordered after the forward edges, if a vertex has no forward edge, then the backward edges are assigned to the other vertex of the edge after the forward edges of that vertex
- within the same vertex, the edges are ordered by the number of the vertex they are connected to

We can establish an ordering between two DFS codes, consider:

$$\begin{aligned} t_1 &= \langle v_i, v_j, \lambda_V(v_i), \lambda_V(v_j), \lambda_E((v_i, v_j))\rangle \\ t_2 &= \langle v_k, v_l, \lambda_V(v_k), \lambda_V(v_l), \lambda_E((v_k, v_l))\rangle \end{aligned} \tag{4.1}$$

Definition 4.7. We say that t_1 is smaller than t_2 ($t_1 < t_2$) if and only if one of the following conditions holds:

- $(v_i, v_j) <_e (v_k, v_l)$
- $(v_i, v_j) = (v_k, v_l)$ and $\langle \lambda_V(v_i), \lambda_V(v_j), \lambda_E((v_i, v_j)) \rangle <_l \langle \lambda_V(v_k), \lambda_V(v_l), \lambda_E((v_k, v_l)) \rangle$

Where $<_e$ is an ordering between edges and $<_l$ is the classical lexicographic ordering between tuples.

Definition 4.8. Given $e_{ij} = (v_i, v_j)$ and $e_{kl} = (v_k, v_l)$, we say that $e_{ij} <_e e_{kl}$ if and only if:

- If they are both forward edges, then if ($j < l$) or ($j = l$ and $i > k$)
- If they are both backward edges, then if ($i < k$) or ($i = k$ and $j < l$)
- If e_{ij} is a forward edge and e_{kl} is a backward edge, then if $j \leq l$
- If e_{ij} is a backward edge and e_{kl} is a forward edge, then if $i \leq l$

Definition 4.9. The **canonical DFS code** of a graph G is the minimum DFS code among all the DFS codes of G.

Definition 4.10. Given two graphs G and G', G is **isomorphic** to G' if and only if their canonical DFS codes are the same.

4.4.2 gSpan algorithm

The gSpan algorithm is based on the pattern growth method, but it uses the DFS code to avoid the generation of duplicates. The algorithm can be described as follows:

Algorithm 10 gSpan algorithm

```
1: Input: DFS code of the pattern being grown s, dataset of graphs D, minimum
   support threshold θ, current set of frequent graphs S
2: Output: Set S of frequent itemsets
3: if s ≠ dfs(s) then
4:     return
5: end if
6: S = S ∪ {s}
7: C = ∅
8: Scan D once, find all the edges e such that s can be rightmost extended to s ◇_r e
9: add s ◇_r e to C and count its frequency
10: sort C in DFS lexicographic order
11: for each frequent s ◇_r e ∈ C do
12:     gSpan(s ◇_r e, D, θ, S)
13: end for
14: return S
```

4.5 Mining variant

Pattern-growth graph mining algorithms can be addapted to handle different types of graphs, such as:

- **Mining unlabeled or partially labeled graphs:** Adding a new empty label ϕ for each unlabeled edge
- **Mining non-simple graphs (i.e., graphs with self-loop or multiple edges):** Adapt gSpan, by adapting the growing order to "backward, self-loop, forward". For the multiple edges, the lexicographic order can handle it.
- **Mining directed graphs:** Adapt gSpan by adding a new element to the state giving the direction, growing the pattern now includes more possibilities
- **Mining disconnected graphs (first case, database contains disconnected graphs):** Adding an edge with an empty label linking the disconnected component.
- **Mining disconnected graphs (second case, mining disconnected patterns):** As a disconnected graph is a set of connected graphs, gSpan can be modified to handle a set of minimum DFS code
- **Mining trees:** If gSpan is set to start at the root, it can take advantage of the tree structure

Like in the previous sections, we can also add constraints to the graph mining process. For example:

- **(Set of) Subgraph(s) containment constraints:** requiring that the mined subgraph contain a given (set of) subgraph(s).
- **Geometric constraints:** constraints on the angles betweens edges in the graph
- **Value-sum constraints:** constraints on the sum of the values of the edges

Classification

Classification is the data analysis task where a model (or **classifier**) is constructed to **predict class** (categorical) labels. Classification is done in two steps:

- **Learning**: We have a learning set where the class labels are known, and we use this set to learn a model that can predict the class labels. The learning step is also called the training step, and the learning set is also called the training set.
- **Classification**: the model is used to predict the class label of data tuples in the test set. Concretely, it is a testing phase of the model previously learned. The test set is a set of data tuples where the class labels are unknown, and we want to predict them.

The test set is used to prevents **overfitting**, which is a situation where the model is too complex and fits the training data too well, but does not generalize well to new data. There exists different types of learnings:

- **Supervised learning**: All of the class labels are known for the data tuples in the learning set.
- **Unsupervised learning**: All of the class labels are unknown for the data tuples in the learning set. And sometimes even some values of the attributes are unknown.
- **Semi-supervised learning**: When only parts of the class labels are known for the data tuples in the learning set.

5.1 Rule-based and Pattern-based classification

A **rule-based classifier** uses a set of **IF-THEN rules** for classification. The **IF** part is called the **rule antecedent** or **precondition**. The **THEN** part is the **rule consequent** and contains the classification.
For a given data tuple, if the condition in a rule antecedent holds true, we say that the rule antecedent is **satisfied** and that the rule **covers** the tuple. For that we need to define the **coverage** and the **accuracy** of a rule:

Definition 5.1. The **coverage** of a rule is the percentage of data tuples in the learning set that satisfy the rule antecedent.

$$\texttt{coverage}(R) = \frac{n_{covers}}{|D|} \tag{5.1}$$

Definition 5.2. The **accuracy** of a rule is the percentage of data tuples that satisfy the rule antecedent and are correctly classified by the rule.

$$\texttt{accuracy}(R) = \frac{n_{correct}}{n_{covers}} \tag{5.2}$$

With these definition, now we can understand how the different rules are chosen.

Definition 5.3. A rule is said to be **triggered** if it satisfies a minimum coverage threshold θ and a minimum accuracy threshold ϵ.

If only one rule is triggered then we say that the rule **fires** by returning a class prediction. But if multiple rules are triggered, we need to use a **conflict resolution strategy** to choose which rule will fire. There is multiple strategies to resolves that.

Definition 5.4. Rule-based ordering is a strategy where all of the possible rules are organized into a priority list before starting the algorithm, the priority list can be based on various criteria such as rule accuracy or coverage or even expert advice. It is known as the **decision list** strategy.

Definition 5.5. Class-based ordering is a strategy where the rules are organized based on the label they predict. For example, we can sort the rules based on a **missclassification cost** or by decreasing importance of the labels.

If no rules fires, we can use a **default rule**. This rules can predicts, for example, the most frequent class or an "unknown" class, this "unknown" class can be interesting as not knowing something this information could be an information itself.

5.1.1 Decision tree to rule based

The rule based classification, can be seen as a decision tree if those rules are:

- **Mutually exclusive**: we cannot have two rules that can fire at the same time because we have one rule for each tuples, so no conflict resolution strategy is needed.
- **Exhaustive**: there is one rule for each tuple, so no default rule is needed.
- **Unordered**: because the rules are mutually exclusive.

With these rules, we can build a decision tree, where each leaf corresponds to a rule, and the path from the root to the leaf correspond to the rule antecedent. And the class label predicted by the rule is the class label of the leaf.
Based on this decision tree structure, for a given rule antecedent, any condition that does not improve its accuracy can be pruned, and thus it generalises the rule.

5.1.2 Sequential covering algorithm

The sequential covering algorithm is a method to learn a set of rules sequentially where each rues for a given class will ideally cover many of the class tuples.

Algorithm 11 SequentialCovering algorithm

Input: Dataset of class-labeled tuples $\mathcal{D}$
Output: Set of rules S
$S = \emptyset$
for each class c **do**
    while not termination condition **do**
        rule = `learn_one_rule`($\mathcal{D}$, attributes ,c)
        Remove covered tuples from $\mathcal{D}$
        $S = S \cup \{rule\}$
    end while
end for
return S

Finding the optimal rule set grows exponentially so to simplify that, we can use a greedy DFS or beam search of size k (keep the k best rules).
To analyse the quality of the rules, we can define some measures.

Definition 5.6. The **entropy** is defined as:

$$H(D) = -\sum_{i=1}^{n} p_i \log_2(p_i) \tag{5.3}$$

with p_i the probability of class i in the dataset D.

Definition 5.7. The **information gain** measures the reduction in entropy caused by adding a new condition to a rule.

Definition 5.8. The **FOIL gain** compute the gain of adding a new rule, this favors rules that cover more positive tuples and have high accuracy, it is defined as:

$$\texttt{FOIL}_{gain}(R, R') = pos' \left(\log_2 \left(\frac{pos'}{pos' + neg'} \right) - \log_2 \left(\frac{pos}{pos + neg} \right) \right) \tag{5.4}$$

with pos and neg the number of positive and negative tuples covered by the rule before adding the new condition, and pos' and neg' the number of positive and negative tuples covered by the rule after adding the new condition.

We can also define pruning rule like `FOIL`.

Definition 5.9. The **FOIL_prune** is defined as:

$$\texttt{FOIL}_{prune}(R) = \frac{pos - neg}{pos + neg} \tag{5.5}$$

Its value increase with the accuracy of R on a prunning set.

5.1.3 Associative classification

When mining association rules for classification, we are looking for association rules of the form $p_1 \wedge p_2 \wedge \ldots \wedge p_n \Rightarrow Class$. With this, rule accuracy is the same as the confidence of the rule. To mine those rules, we need to do a few steps:

- Mine the data for frequent itemsets
- Analyses the frequent itemsets and generates association rules per class that satisfy a minimum confidence and a minimum support threshold
- Organize the rules into a classifier (e.g., by using a decision list strategy)

5.2 Bayesian Belief Network

Bayesian Belief Networks are probabilistic graphical model representing variables and their conditional dependencies using a Directed Acyclic Graph. The probabilities of a node depends on the values of the parents of the nodes. For example, in the figure 5.1, we have a network with 3 nodes, and knowing the probabilities of the parents of the nodes (A and B), we can compute the probabilities of the child node C.
Each nodes represents a variables, and the tables are called **conditional probability tables**. Each tables represents a set of rules and their probabilities reprenses their confidence. For example, in the figure 5.1, we have this rule:

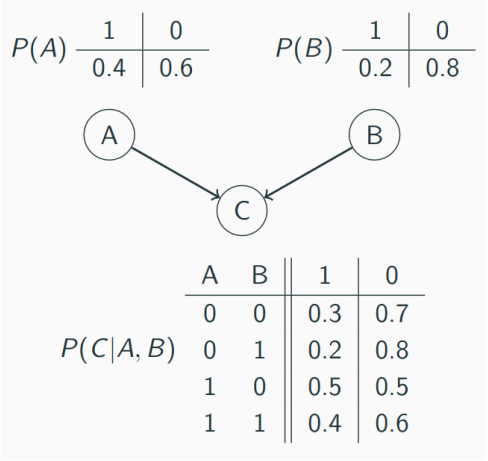


Figure 5.1: Example of Bayesian Belief Network

$$(A = 0) \wedge (B = 0) \Rightarrow C = 0 \text{ with a confidence of } 0.7 \tag{5.6}$$

We can easily generate data from such networks by sampling the parents nodes to get their probabilities then given the samples of the parents, sample the childs.

Definition 5.10. An **evidence** ($E = \overrightarrow{e}$) consists of a set of variables and an assignment $\overrightarrow{e}$ to those variables assumed given.

Given a set of variables X and an evidence $E = \overrightarrow{e}$, the most probable assignment for the variables X is given by:

$$\arg\max_x P(X = x | E = \overrightarrow{e}) \tag{5.7}$$

This is used to predict missing values in data.

5.2.1 Learning a BBN

To learn a Bayesian Belief Network, we need to learn both the structure of the network and the parameters of the network. To do that, we want to maximize the log-likelihood of the parameters given the data and structure. Here is the algorithm to learn a BBN:

1. Initialize the graph without any edges, with each node representing a variable.
2. Compute S, the log-likelihood of the data
3. Repeat until S does not increase anymore:

(a) Add, delete or reverse and edge in G (without creating cycles)

(b) Estimate probabilities given the new structure

(c) Compute the max log-likelihood S'

(d) If S' > S then update best so far as the new graph

5.3 Model evaluation and selection

5.3.1 Metrics

To evaluate the performance of a classification model, we can use different metrics but first we need to define some terms:

- **True Positive (TP)**: the number of positive tuples that are correctly classified as positive
- **True Negative (TN)**: the number of negative tuples that are correctly classified as negative
- **False Positive (FP)**: the number of negative tuples that are incorrectly classified as positive
- **False Negative (FN)**: the number of positive tuples that are incorrectly classified as negative

Confusion matrix:

actual class \ predicted class	yes	no	total
yes	TP	FN	P
no	FP	TN	N
total	P'	N'	P+N

Figure 5.2: Confusion Matrix

From that, we can define the confusion matrix, which is a table that summarizes the performance of a classification model by showing the number of true positives, true negatives, false positives and false negatives.
With these, we can define the accuracy and the error rate of a classification model:

Definition 5.11. The **accuracy** of a classification model is the percentage of correctly classified tuples, it is defined as:

$$\text{accuracy} = \frac{TP + TN}{P + N} \tag{5.8}$$

Definition 5.12. The **error rate** of a classification model is the percentage of incorrectly classified tuples, it is defined as:

$$\text{error rate} = 1 - \text{accuracy} = \frac{FP + FN}{P + N} \tag{5.9}$$

Those metrics are useful when the classes are balanced, but when the classes are imbalanced, we need to introduce new metrics:

Definition 5.13. The **sensitivity** (or **true positive rate**) of a classification model is the percentage of positive tuples that are correctly classified as positive, it is defined as:

$$\text{sensitivity} = \frac{TP}{P} \tag{5.10}$$

Definition 5.14. The **specificity** (or **true negative rate**) of a classification model is the percentage of negative tuples that are correctly classified as negative, it is defined as:

$$\text{specificity} = \frac{TN}{N} \tag{5.11}$$

Definition 5.15. The **precision** of a classification model is the percentage of tuples that are classified as positive that are actually positive, it is defined as:

$$\text{precision} = \frac{TP}{TP + FP} \tag{5.12}$$

Definition 5.16. The **recall** of a classification model is the percentage of positive tuples that are correctly classified as positive, it is defined as:

$$\text{recall} = \frac{TP}{TP + FN} = \frac{TP}{P} \tag{5.13}$$

Definition 5.17. The **F_β-score** is the harmonic mean of precision and recall, it is defined as:

$$F_\beta = \frac{(1 + \beta^2)\text{precision} \times \text{recall}}{\beta^2 \times \text{precision} + \text{recall}} \tag{5.14}$$

β is a parameter that determines the weight of precision and recall. Usually, we use $\beta = 1$ to give equal weight to precision and recall, and thus we get the F1-score. But we $\beta = 0.5$ and $\beta = 2$ are also used to give more weight to precision and recall respectively.

Those metrics are useful to evaluate the performance of a classification model, but those are numeric metrics, and sometimes we need to evaluate the performance of a classification model in a more qualitative way. So we can define some other metrics:

- **Speed**: computational cost involved in generating and using the given classifier.
- **Robustness**: ability to make correct prediction given noisy data or data with missing values. Evaluated on series of syntetic sets with increasing degrees of noise and missing values.
- **Scalability**: ability to construct the classifier efficiently given large amounts of data. Evaluated on increasing size training sets.
- **Interpretability**: level of understanding and insight that is provided by the classifier or predictor. Subjective metric.

5.3.2 Division of the dataset

In order to train the model and evaluate its performance, we need to divide the dataset into multiple sets. There exists different strategies to do that:

- The **holdout** method consists of randomly partitioned into two independent sets (training and testing sets). Typically, two-third are used for training and the rest for testing.
- **Random subsampling** is a variation of the holdout in which the holdout method is repeated k times. The overall accuracy estimate is taken as the average of the accuracies on each iteration.
- In **k-fold cross-validation**, the data are partitionned into k mutually exclusive subsets or "folds" $D_1, D_2, \dots D_k$, each of approximately equal size. Training and testing are performed k times. In iteration i, partition D_i is the test set and the other partitions form the train set.
- **Leave-one-out cross-validation** is a special case of k-fold cross-validation where k is set to the number of initial tuples.
- In **stratified cross-validation** the folds are stratified so that the class distribution of the tuples in each fold is approximately the same as that in the initial data.

5.3.3 Model selection

Given two models M_1 and M_2, trained on the same dataset with a k-fold cross-validation, we can compare them using a student's t-test.

Definition 5.18. The **student's t-test** is a statistical test used to determine if there is a significant difference between the means of two groups. Let $err(M_j)_i$ is the error of model M_j on fold i. We first need to compute the mean error of each model:

$$\overline{err}(M_j) = \frac{1}{k} \sum_{i=1}^{k} err(M_j)_i \tag{5.15}$$

Then we can compute the variance of the errors between the two models:

$$Var(M_1 - M_2) = \frac{1}{k-1} \sum_{i=1}^{k} (err(M_1)_i - err(M_2)_i - (\overline{err}(M_1) - \overline{err}(M_2)))^2 \tag{5.16}$$

Finally, we can compute the t-statistic:

$$t = \frac{\overline{err}(M_1) - \overline{err}(M_2)}{\sqrt{Var(M_1 - M_2)/k}} \tag{5.17}$$

Then we can compare the t-statistic to a critical value from the t-distribution with $k - 1$ degrees of freedom to determine if the difference in performance between the two models is statistically significant. (See table of the t-distribution)

We can also draw the **ROC curves** (Receiver Operating Characteristic) which represents visually the trade-off between the true positive rate and the false positive rate. The closer the ROC curve is to the diagonal, the less accurate the model is.

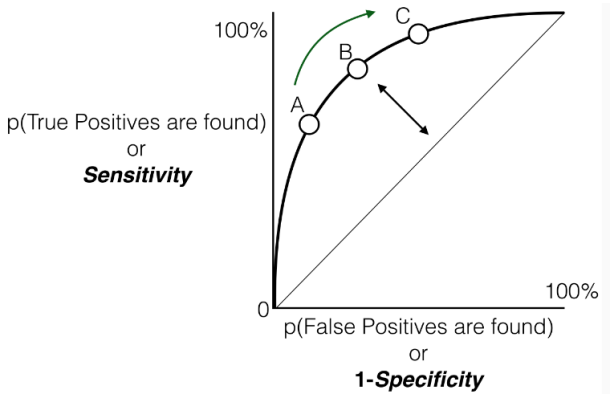


Figure 5.3: ROC curve

5.4 Feature selection

To filter the features, we can use different methods. A feature is good if it is highly correlated with the class label we want to predict. We can use χ^2 test for categorical features or binarized continuous features. To measure the correlation between a feature and the class label, we can use the Fisher score.

Definition 5.19. The **Fisher score** of a feature is defined as:

$$\texttt{Fisher}(f) = \frac{\sum_{i=1}^{n} n_i(\mu_i - \mu)^2}{\sum_{i=1}^{n} n_i \sigma_i^2} \tag{5.18}$$

with n_i the number of training tuples of class i, μ_i the mean of the feature values for class i, μ the mean of the feature values for all classes, and σ_i^2 the variance of the feature values for class i.

We can also use the Entropy score defined previously to compute the information gain.

Definition 5.20. Given $H(y)$, the entropy of the class label y , and $H(y|x)$, the entropy of class label y given the feature x, the information gain can be computed as:

$$\texttt{Information Gain}(x) = H(y) - H(y|x) \tag{5.19}$$

A feature with a larger information gain will reduce more the entropy of the class label y.

Knowing that we can smartly select features, we can introduce **wrapper methods**. Those combines feature selection and classifier training. It is an iterative process where at each iteration, we have a current set of selected features, and we train a classifier on those features, then we evaluate the performance of the classifier and use that performance to decide which features to add or remove from the current set of selected features. This process is repeated until we reach a stopping criterion.

We can do **stepwise forward selection** where we start with an empty set of features and at each iteration, we add the feature that gives the best performance improvement. We can also do **stepwise backward elimination** where we start with all the features and at each iteration, we remove the feature that gives the best performance improvement.

5.5 Classification with weak supervision

In weak supervision, we have a learning set where the class labels are not known for all the data tuples, it is thus a semi-supervised learning. To deal with that, we can use different methods:

- **Self-training**: we train a classifier using the known class labels, then we use that classifier to predict the unknown class labels. The predicted tuples with the highest confidence are added to the training set, and the process is repeated until we reach a stopping criterion.
- **Co-training**: is when two or more classifiers teach each other. Each learner uses a different and ideally independent set of features. The tuples with the most confidence prediction of one model are added to the training set of the second model.

Semi-supervised learning is based on 2 assumptions:

- **cluster assumption**: data tuples tends to organize in clusters.
- **manifold assumption**: data tuples that are close to each other are likely to have the same class label.

But how can select the tuples to label ? Two strategies can be used:

- **uncertainty sampling**: tuples with the most uncertainty are chosen
- **query by committee**: simultaneous training of multiple models, the tuples chosen are the ones that the models disagrees on

To overcome the difficulty of having a small amount of labeled data, we can use different techniques such as:

- **Transfert learning:** We use a trained model on a related task as a starting point for training a model on the target task which has less labeled data.
- **Distant supervision:** We use information in the data to automatically generate labeled data, it is faster than manual labeling. For example if we want to label tweets as positive or negative, we can use the presence of certain words or emojis to automatically label the tweets.
- **Zero-shot learning:** We want to learn a class label never seen in the training data. To do that, we need to learn characteristics instead of labels, and then we can use those characteristics to predict the class label of the unseen class. For example, given a model trained to classify ”dog”, ”owl” and ”fish”. The classifier predicts the concepts ”has four legs”, ”has wings”, ”has retractable claws” and ”has super night vision”. Dog is predicted when ”has four legs” is the only one of the predicted concept. To predict cat, the unknown class, corresponds to all but ”has wings”.

5.6 Classification with rich data type

In **stream data classification**, the classifier needs to be able to classify the new transactions upon arrival and re-train with the new labeled data. To do that, we can use multiples methods. One of them is to break the stream into chunks and train a classifier on each chunk and keep the k historically relevant chunks.
For **sequential data classification**, we can transform them into traditionnal vectors to apply traditionnal predictors on it like n-grams or frequent sequence mining.
For **graph data classification**, we can either classify at graphs level or at node level. And we need to extract features from the nodes.

Clustering

First let us define what is clustering:

Definition 6.1. Clustering is the process of grouping a set of data objects into multiple groups (or clusters) so that objects within a cluster have high similarity, but are dissimilar to objects in other clusters.

Similarity and dissimilarity are often based on distance measures on the attributes values.

6.1 K-means and variants

K-means and its variants are what we call **partitional methods** because they partition the data into k clusters. They are based on the concept of **centroid**, which is the mean of the data points in a cluster, and this centroid can be interpreted as the representative of the cluster.
The quality of a cluster C_i can be measured by the **within-cluster variation** with the centroid c_i defined as:

$$E(C_i) = \sum_{i=1}^{k} \sum_{x \in C_i} d(x, c_i)^2 \tag{6.1}$$

6.1.1 K-means

The K-means algorithm is an iterative algorithm that starts with k initial centroids chosen randomly from the data points, then it alternates between two steps at each iteration t:

- **Assignment step**: each data point is assigned to the cluster with the closest centroid.
- **Update step**: the centroids are updated by computing the mean of the data points in each cluster.

The algorithm converges when the centroids do not change anymore or when the maximum number of iterations is reached. It is an algorithm simple and with a garanty of convergence to a local minimum. It is sensible to the initial centroids, but it can be warm-started by giving an itinial guess of the centroids. It is also scalable to large datasets as the complexity is $\mathcal{O}(nkt)$, so it is quasi linear in the number of data points n. But it presents some limitations, the number of clusters k needs to be specified in advance, it struggles when the clusters are of different sizes and densities, and it is not robust to outliers.

6.1.2 K-medoids

To overcome the problem of outliers in K-means, we can use K-medoids, which is a variant of K-means that uses the medoid instead of the mean as the representative of the cluster. The medoid is the data point in the cluster that minimizes the sum of distances to all other data points in the cluster. So the quality of a cluster C_i can be measured by the **within-cluster variation** with the medoid o_i defined as:

$$E(C_i) = \sum_{i=1}^{k} \sum_{x \in C_i} d(x, o_i)^2 \tag{6.2}$$

The K-medoids algorithm likes this:

- **Initialization step**: choose k initial medoids randomly from the data points.
- **Repeat until convergence**:
  - **Assignment step**: each data point is assigned to the cluster with the closest medoid.
  - **Select and swap step**: select a new medoid o_{nr} and compute the total cost of swapping o_{nr} with each of the current medoids. If one of the swaps improves the total cost, then perform the swap and update the medoids. Otherwise, keep the current medoids.

6.1.3 K-modes

K-modes is a variant of K-means that is used for clustering categorical data and not numerical data. Instead of using the mean as the representative of the cluster, we use the mode, which is the most frequent value of each attribute in the cluster. The quality of a cluster C_i can be measured by the **dissimilarity** between the data points. The dissimilarity between two data points is the number of attributes that have different values. K-modes algorithm objectives is to minimize the total dissimilarity between the data points and the mode of their cluster.

6.1.4 Estimating the number of clusters

To improve the performance of K-means and its variants, we can try to estimate the number of clusters k instead of giving it in advance. For that, we can use the **Calinski-Harabasz index**.

Definition 6.2. The **Calinski-Harabasz index** is defined as:

$$CH(k) = \frac{B(k)/(k-1)}{W(k)/(n-k)} \tag{6.3}$$

with $B(k)$ the between-cluster variation and $W(k)$ the within-cluster variation. The between-cluster variation is defined as:

$$B(k) = \sum_{i=1}^{k} n_i \|c_i - \bar{c}\|^2 \tag{6.4}$$

with n_i the number of data points in cluster C_i, c_i the centroid of cluster C_i, and $\bar{c}$ the centroid of the centroids. And the within-cluster variation, defined as:

$$W(k) = \sum_{i=1}^{k} \sum_{x \in C_i} \|x - c_i\|^2 \tag{6.5}$$

The optimal number of clusters k is the one that maximizes the Calinski-Harabasz index.

6.2 Hierachical methods

Definition 6.3. A **hierarchical method** creates a **hierarchical decomposition** of a given set of data objects.

There are two types of hierarchical methods, the **agglomerative** and the **divisive** methods.

Definition 6.4. An **agglomerative hierarchical** clustering method uses a **bottom-up** strategy. It starts with each object is its own cluster and **iteratively merges** clusters until either there is only one cluster left, or some conditions are reached.

Definition 6.5. A **divisive hierarchical** clustering method employs a **top-down** strategy. It starts by placing every object in one cluster, **recursively partitions** clusters into smaller one until every items has their own clusters, or some conditions are reached.

To choose how to merge or split clusters, we can use different measures based on the distance between the elements of the clusters.

Definition 6.6. The **neirest-neighbor clustering** algorithm (or **single-linkage** algorithm) uses the minimum distance to measure the distance between clusters

$$dist_{min}(C_i, C_j) = \min_{p \in C_i, p' \in C_j} \{\|p - p'\|\} \tag{6.6}$$

Definition 6.7. The **farthest-neighbor clustering** algorithm (or **complete-linkage** algorithm) uses the maximum distance to measure the distance between clusters

$$dist_{max}(C_i, C_j) = \max_{p \in C_i, p' \in C_j} \{\|p - p'\|\} \tag{6.7}$$

To be less sensitive to outliers, we can also define:

Definition 6.8. The **average-linkage** algorithm uses the average distance to measure the distance between clusters

$$dist_{average}(C_i, C_j) = \frac{1}{n_i n_j} \sum_{p \in C_i} \sum_{p' \in C_j} \{\|p - p'\|\} \tag{6.8}$$

Definition 6.9. The **mean-linkage** algorithm uses the distance between the centroids to measure the distance between clusters

$$dist_{mean}(C_i, C_j) = \|m_i - m_j\| \tag{6.9}$$

We need to define a criterion to stop the merging or splitting of clusters.

Definition 6.10. Ward's criterion looks at the increase of the within-cluster variation:

$$\begin{aligned} W(C_i, C_j) &= \sum_{x \in C_i \cup C_j} \|x - m_{ij}\|^2 - \sum_{x \in C_i} \|x - m_i\|^2 - \sum_{x \in C_j} \|x - m_j\|^2 \\ &= \frac{n_i n_j}{n_i + n_j} \|m_i - m_j\|^2 \end{aligned} \tag{6.10}$$

For the divisive hierarchical clustering, we can solve this problem with a minimum spanning tree based approach. This can be solved with Prim's or Kruskal's algorithm. It can be seen like this:

- Given a set of points, we can construct a weighted graph such that each point is represented by a node in the graph, and the distance between two points is the weights of the edge connecting the two corresponding nodes in the graph.
- A bisecting splitting can be done by splitting one cluster into two by removing the edge with the largest weight.

Then we can represent visualy this hierarchical clustering using a **dendrogram**.

6.3 Density-based and Grid-based methods

Definition 6.11. Density-based clustering have been developed based on the notion of density. Their general idea is to continue growing a given cluster as long as the density (number of data points) in the neighborhood exceeds some threshold.

Definition 6.12. Grid-based clustering is based on the fact that we can put all the data points in an finite number of cells forming a uniform grid structure. The clustering operations are performed on the grid structure (i.e., cells are used to assemble clusters).

The **density** of an object o is measured by the number of objects in its neighborhood. **Core objects** are the objects that have a high density.

6.3.1 DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) connects core objets and their neighborhoods to form dense regions as clusters. To understand well DBSCAN algorithm, we need to define some concepts.

Definition 6.13. An object p is a **core object** if the number of objects in its ϵ-neighborhood is greater than or equal to $minPts$.

Definition 6.14. Two core objects p and q are **ϵ-reachable** if $d(p, q) \leq \epsilon$, that is p is in the ϵ-neighbor of q and vice versa.

Definition 6.15. Two core objects p and q are transitively ϵ-reachable if there exist one or multiple core objects $r_1, \ldots, r_l$ such that p and r_1 are ϵ-reachable, r_k and r_{k+1} are ϵ-reachable (for $1 \leq k < l$), and r_l and q are ϵ-reachable.

Definition 6.16. Two core objects p and q are **density-connected** if p and q are ϵ-reachable or transitively ϵ-reachable.

With these definitions, we can now understand the DBSCAN algorithm. Which is described below:

Algorithm 12 DBSCAN

Input: Dataset of class-labeled tuples $\mathcal{D}$, radius if a neighborhood ϵ, minimum number of points in a neighborhood $minPts$
Output:
mark all object as unvisited
while no object is unvisited **do**
    randomly select an unvisited object p and mark it visited
    if p is a core object **then**
        create new cluster C with object p in it
        $N =$ the ϵ-neighborhood of p
        for each point p' in N **do**
            if p' is unvisited **then**
                mark p' as visited
                if if p' is a core object **then**
                    add the ϵ-neighborhood of p' to N
                end if
            end if
        end for
    else
        mark p as noise
    end if
end while

6.3.2 HDBSCAN

HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise) it takes only one parameter $minPts$. It is based on the concept of **mutual reachability distance** defined as:

$$d_{mreach}(p,q) = \max\{d_{core}(p), d_{core}(q), d(p,q)\} \tag{6.11}$$

with $d_{core}(p)$ the **core distance** of p defined as the distance between p and its $minPts$-th nearest neighbor. Then, we can define the **mutual reachability graph**, it is complete graph where every object is a node and weight of the edge between p and q is the mutual reachability distance $d_{mreach}(p,q)$.

6.3.3 CLIQUE

CLIQUE (Clustering In QUEst) is a grid-based clustering algorithm that is based on the monotinicity between dimensions. It means that a k-dimensional cell c $(k > 1)$ can have at least l points only if every $(k-1)$-dimensional projection of c, which is a cell in a $(k-1)$-dimensional subspace, has at least l points. And it's algorithm is as follows:

- Step 1: Partitionning of the d-dimensional sata space into nonoverlapping rectangular units
- Step 2: identifies the dense units among the dimensions. First, for each dimension identify the dense intervals, then join the dense one together to form higher dimension cells
- Step 3: assemble clusters together

6.4 Evaluation of clustering

A clustering algorithm always produces a clustering result, but these results are not always good. So we need to evaluate the quality of the clustering result.

6.4.1 Assessing clustering tendency

Definition 6.17. The **Hopkins statistic** is a measure of the clustering tendency of a dataset. It is defined as:

$$H = \frac{\sum_{i=1}^{n} x_i^d}{\sum_{i=1}^{n} x_i^d + \sum_{i=1}^{n} y_i^d} \tag{6.12}$$

if the points are uniformly distributed, H tends to be around 0.5. If D is highly skewed, then H tends to be close to 1. Where x_i is the distance from a randomly selected point in the dataset to its nearest neighbor, and y_i is the distance from a randomly generated point in the same space to its nearest neighbor in the dataset.

$$x_i = \min_{v \in D} \{dist(p_i, v)\} \tag{6.13}$$

$$y_i = \min_{v \in D, v \neq q_i} \{dist(q_i, v)\} \tag{6.14}$$

6.4.2 Measuring clustering quality

To measure the quality of a clustering result, we can define some metrics:

Definition 6.18. The **compactness** (maximum in-cluster distance) of clusters is the maximum distance between two points that belong to the same cluster.

$$\Delta = \max_{C(o_i)=C(o_j)} \{dist(o_i, o_j)\} \tag{6.15}$$

Definition 6.19. The **degree of separation** (minimum inter-cluster distance) among different clusters is the minimum distance between two points that belong to different clusters.

$$\delta = \min_{C(o_i)\neq C(o_j)} \{dist(o_i, o_j)\} \tag{6.16}$$

From that, we can define the **Dunn index**.

Definition 6.20. The **Dunn index** is defined as:

$$D = \frac{\delta}{\Delta} \tag{6.17}$$

A higher Dunn index indicates better clustering results, as it means that the clusters are well separated and compact. The Dunn index is sensitive to noise and outliers.

We can also define the **silhouette coefficient**. But first, we need to define two concepts:

Definition 6.21. $a(o)$ is the average distance from o ($o \in C_i$) to all points in the same cluster (C_i):

$$a(o) = \frac{\sum_{o' \in C_i, o \neq o'} dist(o, o')}{|C_i| - 1} \tag{6.18}$$

Definition 6.22. $b(o)$ is the minimum average distance from o ($o \in C_i$) to all clusters to which o does not belong (C_j with $j \neq i$):

$$b(o) = \min_{C_j : 1 \leq j \leq k} \{\frac{\sum_{o' \in C_j} dist(o, o')}{|C_j|}\} \tag{6.19}$$

And with that, we have:

Definition 6.23. The **silhouette coefficient** is defined as:

$$s(o) = \frac{b(o) - a(o)}{\max\{a(o), b(o)\}} \tag{6.20}$$

it lies between -1 and 1. A value close to 1 indicates that the object is well clustered, a value close to 0 indicates that the object is on the border of two clusters, and a value close to -1 indicates that the object is possibly misclassified.

The cluster fitness is defined as the average silhouette coefficient value of all objects in the cluster.

6.5 Fuzzy clustering

Until now clustering was focused on assigning each data point to exactly one cluster, but in some cases, it is more appropriate to allow data points to belong to multiple clusters. This is called **fuzzy or flexible clustering**. A fuzzy clustering of k fuzzy clusters ($C_1, \ldots, C_k$) can be represented using a partition matrix $M = [w_{ij}](1 \leq i \leq n, 1 \leq j \leq k)$, where w_{ij} is the degree of membership of data point o_i to cluster C_j. The matrix M must satisfy the following conditions:

- $w_{ij} \in [0, 1] \; \forall i, j$
- $\sum_{j=1}^{k} w_{ij} = 1 \; \forall i$
- $0 < \sum_{i=1}^{n} w_{ij} < n \; \forall j$

To compute the degree of membership w_{ij}, we can use the following formula:

$$w_{ij} = \frac{1}{\sum_{l=1}^{k} \left(\frac{dist(o_i, c_j)}{dist(o_i, c_l)} \right)^{\frac{2}{p-1}}} \tag{6.21}$$

The centroids of the clusters can be computed as:

$$c_j = \frac{\sum_{i=1}^{n} w_{ij}^p o_i}{\sum_{i=1}^{n} w_{ij}^p} \tag{6.22}$$

To evaluate the quality of a fuzzy clustering result, we can define the sum of squared errors (SSE), with $p(\geq 1)$ the parameter that controls the fuzzyness of the clusters:

- For an object o_i, the sum of squared errors is defined as:

$$SSE(o_i) = \sum_{j=1}^{k} w_{ij}^p dist(o_i, c_j)^2 \tag{6.23}$$

- For a cluster C_j, the sum of squared errors is defined as:

$$SSE(C_j) = \sum_{i=1}^{n} w_{ij}^p dist(o_i, c_j)^2 \tag{6.24}$$

- For the whole clustering result, the sum of squared errors is defined as:

$$SSE = \sum_{i=1}^{n} \sum_{j=1}^{k} w_{ij}^p dist(o_i, c_j)^2 \tag{6.25}$$

6.5.1 Expectation-Maximization algorithm

Definition 6.24. In the context of fuzzy clustering, an **Expectation-Maximization (EM) algorithm** starts with an initial set of parameters and iterates until the clustering cannot be improved (until it converges or changes are sufficiently small).

Each iteration of the EM algorithm consists of two steps:

- **Expectation step**: assigns objects to clusters according to the current fuzzy clustering
- **Maximization step**: finds the new clustering that minimize the SSE

6.6 High dimensionnal data

When the data is high dimensionnal, often we encounter the **curse of dimensionality**, which is the phenomenon that the distance between data points becomes less meaningful as the number of dimensions increases. To overcome this problem, we can use the CLIQUE algorithm.

6.7 Biclustering

The biclustering is a type of clustering that allows to cluster to be in two dimensions. It means that instead of saying that two data points are similar, we can say that two data points are similar with respect to a subset of features.
Here are some examples of the simplest biclusters with a matrix representation $I \times J$:

- **Bicluster with constant values**: all the data points in the bicluster have the same value for a subset of features.
- **Bicluster with constant values on rows**: all the data points in the bicluster have the same value for a subset of features on rows, but they can have different values on columns. Mathematically, we have $e_{ij} = c + \alpha_i$.
- **Bicluster with constant values on columns**: all the data points in the bicluster have the same value for a subset of features on columns, but they can have different values on rows. Mathematically, we have $e_{ij} = c + \beta_j$.
- **Bicluster with coherent values**: all the data points in the bicluster have a pattern that is coherent with respect to the rows and columns. For example, each row has a pattern and columns shift them (i.e., $e_{ij} = c + \alpha_i + \beta_j$). Equivalently, a bicluster is with coherent values if and only if for any $i_1, i_2 \in I$ and $j_1, j_2 \in J$ then $e_{i_1 j_1} - e_{i_1 j_2} = e_{i_2 j_1} - e_{i_2 j_2}$.
- **Bicluster with coherent evolutions on rows**: all the data points in the bicluster have a pattern that evolves coherently on rows, but they can have different values on columns. It means that for any $i_1, i_2 \in I$ and $j_1, j_2 \in J$ then $(e_{i_1 j_1} - e_{i_1 j_2})(e_{i_2 j_1} - e_{i_2 j_2}) \geq 0$, Symmetrically, we can define **bicluster with coherent evolutions on columns**.

But these cases barely happen in real life due to noise and outliers. To find biclusters in real life data, we can either do it with optimization-based methods or with enumeration methods. First, we need to some means for the submatrix $I \times J$:

- The mean of the i^{th} row is defined as: $\bar{e}_{iJ} = \frac{1}{|J|} \sum_{j \in J} e_{ij}$
- The mean of the j^{th} column is defined as: $\bar{e}_{Ij} = \frac{1}{|I|} \sum_{i \in I} e_{ij}$
- The mean of the submatrix $I \times J$ is defined as:

$$\bar{e}_{IJ} = \frac{1}{|I||J|} \sum_{i \in I} \sum_{j \in J} e_{ij} = \frac{1}{|I|} \sum_{i \in I} \bar{e}_{iJ} = \frac{1}{|J|} \sum_{j \in J} \bar{e}_{Ij} \tag{6.26}$$

Definition 6.25. The **quality** of the submatrix as a bicluster can be measured by the mean-square residue value:

$$H(I \times J) = \frac{1}{|I||J|} \sum_{i \in I, j \in J} (e_{ij} - \bar{e}_{iJ} - \bar{e}_{Ij} + \bar{e}_{IJ})^2 \tag{6.27}$$

This quality measure is based on the ideal bicluster with coherent values, because the quality of a bicluster with coherent values is 0.

6.7.1 δ-bicluster algorithm

The idea of the δ-bicluster algorithm is to find a submatrix $I \times J$ representing a bicluster, such that the values are coherent but it allows noise so it finds a bicluster $I \times J$ such that $H(I \times J) \leq \delta$.

Definition 6.26. A **maximal δ-bicluster** is a δ-bicluster $I \times J$ such that there is no δ-bicluster $I' \times J'$ and $I \subseteq I'$ and $J \subseteq J'$ with $I \neq I'$ or $J \neq J'$.

The algorithm works in two steps, first a deletion part where we try to remove the noisy part until a structure appears, then we have an addition part where we try to add more rows and columns to the bicluster until we cannot add more.
In deletion part, we want to remove the row or column with the highest error. This error is called the **mean-square residue** and is defined as:

$$d(i) = \frac{1}{|J|} \sum_{j \in J} (e_{ij} - \bar{e}_{iJ} - \bar{e}_{Ij} + \bar{e}_{IJ})^2 \tag{6.28}$$

$$d(j) = \frac{1}{|I|} \sum_{i \in I} (e_{ij} - \bar{e}_{iJ} - \bar{e}_{Ij} + \bar{e}_{IJ})^2 \tag{6.29}$$

And so for the deletion part, we remove the row or column with the highest mean-square residue until $H(I \times J) \leq \delta$.
Then we have the addition part we add back rows and columns with the lowest mean-square residue as long as $H(I \times J) \leq \delta$.

6.8 Clustering graph and network data

To do clustering on graph and network data, we need to redefine the concept of distance between data points.

Definition 6.27. The **geodesic distance** $d(u, v)$ is the length of the shortest path between two nodes u and v in a graph. With $d(u, v) = +\infty$

Definition 6.28. For a vertex $v \in V$, the **eccentricity** of v, denoted $\texttt{eccen}(v)$, is the largest geodesic distance between v and any other vertex $u \in V - \{v\}$

$$\texttt{eccen}(v) = \max_{u \in V - \{v\}} d(u, v) \tag{6.30}$$

Definition 6.29. The **radius** of a graph G is the minimum eccentricity of any vertex in G:

$$\texttt{radius}(G) = \min_{v \in V} \texttt{eccen}(v) \tag{6.31}$$

Definition 6.30. The **diameter** of a graph G is the maximum eccentricity of any vertex in G:

$$\texttt{diameter}(G) = \max_{v \in V} \texttt{eccen}(v) \tag{6.32}$$

Definition 6.31. A **periferal** vertex is a vertex whose eccentricity is equal to the diameter of the graph.

In some networks, nodes can be considered similar if they have similar neighbors.

Definition 6.32. In a directed graph $G = (V, E)$, the **individual in-neighborhood** of v is defined as:

$$I(v) = \{u|(u, v) \in E\} \tag{6.33}$$

Definition 6.33. In a directed graph $G = (V, E)$, the **individual out-neighborhood** of v is defined as:

$$O(v) = \{u|(v, u) \in E\} \tag{6.34}$$

Definition 6.34. The **SimRank** similarity measure is based on the idea that two nodes are similar if they are related to similar nodes. For vertices $u, v \in V$ with $u \neq v$, it is defined as:

$$s(u, v) = \frac{C}{|I(u)||I(v)|} \sum_{x \in I(u)} \sum_{y \in I(v)} s(x, y) \tag{6.35}$$

with C a constant between 0 and 1, representing the decay factor of the similarity score. $s(v, v) = 1$ for all $v \in V$. and $s(u, v) = 0$ if either $I(u)$ or $I(v)$ is empty.

Definition 6.35. Given an undirected graph $G = (V, E)$, for a vertex $u \in V$, the neighborhood of u is $\Gamma(u) = \{v|(u, v) \in E\} \cup \{u\}$.

Definition 6.36. SCAN (Structural Clustering Algorithm for Networks) measures the similarity between two vertices, $u, v \in V$

$$\sigma(u, v) = \frac{|\Gamma(u) \cap \Gamma(v)|}{\sqrt{|\Gamma(u)||\Gamma(v)|}} \tag{6.36}$$

The larger the value, the most similar the nodes.

Definition 6.37. For a vertex, $u \in V$, the **ϵ-neighborhood** of u is defined as

$$N_\epsilon(u) = \{v \in \Gamma(u)|\sigma(u, v) \geq \epsilon\} \tag{6.37}$$

$u \in V$ is a core vertex if $|N_\epsilon(u)| \geq \mu$, where μ is a popularity threshold.

If a vertex v is in the ϵ-neighborhood of a core u, then v is assigned to the same cluster as u.

6.9 Semisupervised clustering

6.9.1 Constrained k-means

The constrained k-means is a variant of k-means. We have some prior knowledge about the data, and we want to use this knowledge to improve the clustering result. We have labeled subset S and know that each S_j contains points already known to belong to cluster C_j. We know that these subsets are disjoint. The algorithm assigns the labeled points to their corresponding clusters and then assigns the unlabeled points to the closest cluster. The algorithm then updates the centroids of the clusters and repeats until convergence. The labeled points will anchor their corresponding clusters.

6.9.2 COP-k-means algorithm

For the COP-k-means algorithm, we need to define **pairwise constraint** such as:

Definition 6.38. Must-link constraint is a constraint that specifies that two data points must be in the same cluster.

By transitivity, we have the following relation:

$$\texttt{mustLink}(x,y) \wedge \texttt{mustLink}(y,z) \Rightarrow \texttt{mustLink}(x,z) \tag{6.38}$$

Definition 6.39. Cannot-link constraint is a constraint that specifies that two data points cannot be in the same cluster.

These two constraints are linked and can be propagated together, for example:

$$\texttt{cannotLink}(x,y) \wedge \texttt{mustLink}(x,x') \wedge \texttt{mustLink}(y,y') \Rightarrow \texttt{cannotLink}(x',y') \tag{6.39}$$

The COP-k-means algorithm works as:

1. Choose k objects in D as the initial clusters centers $c_1, \ldots, c_k$ such that there is no must-link between any two of those k objects
2. Repeat until convergence:
   (a) Assign each object to the cluster whose mean is the closest, and without pairwise violation constraints
   (b) Update the cluster means

6.10 Mining compressed or approximate patterns

When we mine patterns, we often get a large number of patterns, and many of them are similar to each other. For example if we have (i_2, i_4, i_5) with a cover of 205 and (i_2, i_4, i_5, i_6) with a cover of 205, we see that the first pattern is redundant, we can thus only keep the second pattern. To find those redundant patterns, we can define some concepts:

Definition 6.40. The **pattern distance** between two patterns P_1 and P_2 is defined as:

$$d(P_1, P_2) = 1 - \frac{|\texttt{cover}_{\mathcal{D}}(P_1) \cap \texttt{cover}_{\mathcal{D}}(P_2)|}{|\texttt{cover}_{\mathcal{D}}(P_1) \cup \texttt{cover}_{\mathcal{D}}(P_2)|} \tag{6.40}$$

Definition 6.41. Given two patterns P_1 and P_2, we say that P_2 can be **expressed** by P_1 if the itemset of P_2 is a subset of the itemset of P_1 ($I(P_2) \subseteq I(P_1)$).

Definition 6.42. Given patterns $P_1, P_2, \ldots, P_k$ in a same cluster. The **representative** P_r of the cluster should be able to express all the patterns in the cluster:

$$\forall i \in \{1, 2, \ldots, k\}, I(P_i) \subseteq I(P_r) \tag{6.41}$$

or

$$\bigcup_{i=1}^{k} I(P_i) \subseteq I(P_r) \tag{6.42}$$

Definition 6.43. A pattern P is **δ-covered** by a pattern P' if $I(P) \subseteq I(P')$ and $d(P, P') \leq \delta$.

Definition 6.44. A set of patterns form a **δ-cluster** if there exists a representative P_r such that for each pattern P in the cluster, P is δ-covered by P_r.

Outlier detection

Definition 7.1. Outlier detection (or **anomaly detection**) is the process of finding data objects with behaviors that are very different from expectation.

Clustering and outliers detections are related tasks, because clustering finds the majority patterns in the dataset, while outlier detection search for points that deviates from the majority patterns. Beware that outlier are different from noise, noise is a random error or variance in a measured variable, while outliers are data points that are significantly different from the rest of the data. Noise is not interesting in data analysis, while outliers can be interesting because they can indicate a rare event or a novel pattern.

7.1 Statistical approaches

Statistical methods for outlier detection make assumptions about data normality, it means that it supposes the data is generated by a stochastic process. And so with that, we can deduce that normal data points will have a high probability of being generated by the stochastic process, while outliers will have a low probability of being generated by the stochastic process.

7.1.1 Parametric methods

A parametric method assumes that the normal data objects are generated by a parametric distribution with a finite number of parameters Θ.

Definition 7.2. The probability density function of the parametric distribution $f(x, \Theta)$ gives the probability that object x is generated by the distribution.

The smaller this value, the more likely x is an outlier. For example for univariate data, we can check using a normal distribution, we would have:

$$\mu = \frac{1}{n} \sum_{i=1}^{n} x_i \tag{7.1}$$

$$\sigma = \frac{1}{n} \sum_{i=1}^{n} (x_i - \mu)^2 \tag{7.2}$$

And we know that the interval $[\mu - 3\sigma, \mu + 3\sigma]$ contains 99.7% of the data points, so any point outside this interval is considered as an outlier. With multivariate data, we

can check using a χ^2-test, under the assumption of normal distribution. We have:

$$\chi^2 = \sum_{i=1}^{n} \frac{(o_i - E_i)^2}{E_i} \tag{7.3}$$

where o_i is the i^{th} dimension of object o and E_i is the expected value of the i^{th} dimension. If χ^2 is large, then o is an outlier. For mixture of two distributions, we can use the following formula:

$$Pr(o|\Theta_1, \Theta_2) = w_1 f_{\Theta_1}(o) + w_2 f_{\Theta_2}(o) \tag{7.4}$$

where w_1 and w_2 are the weights of the two distributions, and $f_{\Theta_1}(o)$ and $f_{\Theta_2}(o)$ are the probability density functions of the two distributions Θ_1 and Θ_2.

7.1.2 Nonparametric methods

A nonparametric method does not assume an a priori statistical model with a finite number of parameters. Instead, it tries to determine the model from the input data. It make fewer assumptions about the data distribution.

Definition 7.3. The **histogram-based method** is a method that divides the data space into a finite number of bins and counts the number of data points in each bin. The bins with a low count are considered as outliers.

This method require an user-specified bin size, and it is sensitive to the choice of the bin size. If the bin size is too small, it could lead to false positives, while if the bin size is too large, it could lead to false negatives.

7.2 Proximity-based Approaches

7.2.1 Distance-based outlier detection

A distance-based outlier detection method consults the neighborhood of an object, which is defined by a radius. An object is then considered as an outlier if its neighborhood does not have enough other points.

Definition 7.4. An object o is a $DB(r, \pi)$**-outlier** if the number of objects in its neighborhood of radius r is less than π percent of the total number of objects in the dataset. Mathematically, we have:

$$\frac{|\{o'|dist(o, o') \leq r\}|}{|D|} \leq \pi \tag{7.5}$$

7.2.2 Density-based outlier detection

Definition 7.5. The **k-distance** of an object o denoted $dist_k(o)$ is the distance from o to its k^{th} nearest neighbor.

Definition 7.6. The **k-distance neighborhood** of an object o is the set of objects that are at a distance less than or equal to $dist_k(o)$.

$$N_k(o) = \{o'|o' \in D, dist(o, o') \leq dist_k(o)\} \tag{7.6}$$

Definition 7.7. For two objects o and o', the **reachability distance** of o with respect to o' is defined as:

$$reachdist_k(o, o') = \max\{dist_k(o'), dist(o, o')\} \tag{7.7}$$

Beware that the reachability distance is not symmetric, it means that $reachdist_k(o, o') \neq reachdist_k(o', o)$.

Definition 7.8. The **local reachability density** of an object o is defined as the inverse of the average reachability distance of o with respect to its k-distance neighborhood:

$$lrd_k(o) = \frac{|N_k(o)|}{\sum_{o' \in N_k(o)} reachdist_k(o, o')} \tag{7.8}$$

This measure allows to compute a local density that can change in various regions of the data space.

Definition 7.9. The **local outlier factor (LOF)** of an object o is defined as:

$$LOF_k(o) = \frac{\sum_{o' \in N_k(o)} \frac{lrd_k(o')}{lrd_k(o)}}{|N_k(o)|} = \sum_{o' \in N_k(o)} lrd_k(o') \times \sum_{o' \in N_k(o)} reachdist_k(o', o) \tag{7.9}$$

The LOF is:

- lower than 1 for items with a higher density than neighbors
- close to 1 for items with a similar density as the neighbors
- higher than 1 for items with a lower density than the neighbors

7.3 Learning-based approaches

7.3.1 Clustering-based approaches

Outliers can be discovered by examining the relationship between objects and clusters. There is multiples ways to know if an object is an outlier with respect to a cluster, for example:

- If an object does not belong to any cluster, then it is an outlier.
- If an object is at an high distance from the closest cluster, then it is an outlier.
- If a small or sparse cluster is formed, then the objects in this cluster are probably outliers.

Definition 7.10. The **distance ratio** measure how the distance of an object o to the center of its closest cluster compares to the average distance of the objects in the cluster to the center of the cluster. It is defined as:

$$DR(o) = \frac{dist(o, c_o)}{\frac{1}{|C|} \sum_{o' \in C} dist(o', c_o)} \tag{7.10}$$

We also have the method **FindCBLOF (Find Cluster-Based Local Outlier Factor)**, it is an algorithm that works as follows:

1. Find clusters in the data
2. Sort the cluster by decreasing size.
3. For each data point assign a $CBLOF$ value:
    - in large cluster: it is the product of the cluster's size and the similarity between the point and the cluster
    - in small cluster: it is the product of the cluster's size and the similarity between the point and the closest large cluster

The lower the $CBLOF$ value, the more probable the point is an outlier.
Those techniques requires to compute all the clusters which is expensive. A solution is to use **fixed-width clustering**, it is less precise but it is computed in linear time. Given a predefined width δ, it works as follows for each data point o:

- If there is a cluster C such that $dist(o, c) \leq \delta$, then assign o to C.
- Otherwise, create a new cluster with o as the only member and o as the center.

7.3.2 Classification-based approaches

It is possible to use classification techniques to detect outliers, by training a classifier to label the data as "normal" or "outlier". But this comes with some potential problems such as highly imbalanced classes, the outliers present in the training data may not be representative of the outliers in the test data.

7.3.3 Semi-supervised approaches

When only few data are labeled, two cases arise:

- Some normal examples are labeled: With the unlabeled data, they can be used to model the normal data.
- Only outliers examples are labeled: Trickier case, as this might not cover the whole distribution of outliers. Unlabeled data can be used to train a model for the normal data.

7.4 Outlier Detection in High-Dimensional Data

As usual the curse of dimensionality makes the distance between data points less meaningful, and so it becomes harder to detect outliers. To overcome this problem, we can use the **HilOut algorithm**. The HilOut algorithm finds distance-based outliers, but use the ranks of distance instead of the distances themselves. The HilOut algorithm works as follows:

- For each objects o, HilOut finds the k-nearest neighbors of o, denoted by $nn_1(o), \ldots, nn_k(o)$, where k is a parameter.

- The weights of the object o is defined as:

$$w(o) = \sum_{i=1}^{k} dist(o, nn_i(o)) \tag{7.11}$$

- sort the objects by decreasing weight, and return the top l objects as outliers, where l is a parameter.

It is also possible to reduce the dimensionality of the data before applying outlier detection techniques such as grid-based methods where we search for subspaces with lower than average density. We will thus be searching for outliers in subspaces. To find such subspace projections, we first discretize the data into a grid in an equal-depth way, i.e., each ϕ range contains a fraction $f = 1/\phi$ of the objects. The sparsity of the subspace is then evaluated. Given a k-dimensional subspace with n objects in total, the expected number of object in each k-dimensional cubes is $(\frac{n}{\phi^k})$

Definition 7.11. Given $n(C)$ the number of object in a cube C, the **sparsity coefficient** of C is:

$$S(C) = \frac{n(C) - f^k n}{\sqrt{f^k(1 - f^k)n}} \tag{7.12}$$

If $S(C) < 0$, then C contains fewer objects than expected. The smaller the value of $S(C)$, the sparser C is and the more likely the objects in C are outliers in the subspace. We can also use **Angle-Based Outlier Detection (ABOD)** which is based on the idea that a point in the center of a cluster will have widely diverse set of angles while a a point outside of a cluster will have a very small variation. Thus for each object o, compute the angle $\widehat{xoy}$ for each pair of point x, y such as $x \neq o$ and $y \neq o$. Then the variance of these angles is computed, and the smaller the variance, the more likely o is an outlier. It can be mathematically expressed as:

$$ABOF(o) = Var_{x,y \in D \setminus \{o\}} \frac{\langle \vec{ox}, \vec{oy} \rangle}{dist(o,x)^2 dist(o,y)^2} \tag{7.13}$$